ZeBu® Virtual Ethernet User Guide

Version W-2025.06, September 2025

SYNOPSYS®

Copyright and Proprietary Information Notice

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at https://www.synopsys.com/company/legal/trademarks-brands.html.
All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Feedback

Contents

Preface

This chapter has the following sections:

About This Book

The *ZeBu® Virtual Ethernet User Guide* describes how to perform comprehensive pre-silicon validation of a Design Under Test (DUT) in a virtual environment using Synopsys ZeBu emulator, ZeBu transactors, and third-party virtual test platforms.

Contents of This Book

The *ZeBu® Virtual Ethernet User Guide* consist of the following sections:

Section	Describes...
Introduction to Virtual Ethernet	Provides a brief overview of Virtual Ethernet solution along with its benefits. It also describes the compatibility of the solution with third-party products.
Host Reconfiguration	Provides steps to reconfigure the host setup.
Install and Setup	Provides steps to install and setup the Virtual Ethernet package apart from other software required for the solution to work.
Configure YAML Fields	Provides a list of YAML fields that need to be configured for the solution to work. It is divided into eight sections.
RPC Server and User API	Lists supported Tcl commands.
Virtual Ethernet Examples	Describes how to compile and run the example that comes with the Virtual Ethernet package.
Third Party Applications and Libraries	Provides steps to install and use third party applications and libraries that make it easy to use Tcl commands.

Feedback

Related Documentation

Document	Description
ZeBu User Guide	Provides a complete reference to the Synopsys emulator system, ZeBu. To request for a copy, contact your Synopsys Account Manager.
ZeBu Virtual System Adaptors Installation Guide	Provides necessary steps to install various VSA solutions, including Virtual Ethernet.
ZeBu Virtual Ethernet Tcl Commands Reference Guide	Describes interactive virtual traffic generator Tcl commands that are used at runtime.
ZeBu Ethernet Transactor User Manual	Provides a list of features of the ZeBu 400G Ethernet transactor, which is the underlying transactor in this solution. To request for a copy, contact your Synopsys Account Manager.
ZeBu Virtual System Adaptors Release Notes	Provides enhancements, new features, and limitations in the current Virtual Ethernet release.
ZeBu Virtual System Adaptors LCA Features Guide	Provides a list of Limited Customer Availability (LCA) features available with Virtual System Adaptors.

Other Useful References

Document Name	Description
IxVerify - Reference Guide	Provides complete information on prerequisites, supported platforms, and initial configuration required for installing IxVerify. It is available on the Ixia website.
IxOS - IxExplorer User Guide	Provides details about IxExplorer operation, features, functions, and available options. It is available on the Ixia website.
IxVerify – Installation over KVM over CentOS	Provides information on installing Virtual Chassis and Virtual Load Modules (VLMs) over KVM (QEMU) over CentOS 7.0 64-bit. It is available on the Ixia website.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary

language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction to Virtual Ethernet

Virtual Ethernet enables seamless integration of third-party network traffic generators with Synopsys ZeBu emulator and ZeBu transactors. In this solution, networking devices from the design under test (DUT) are connected to ZeBu transactors (*xtor_enet_svs*) that are serviced by virtual ports from third-party testers through Virtual Ethernet.

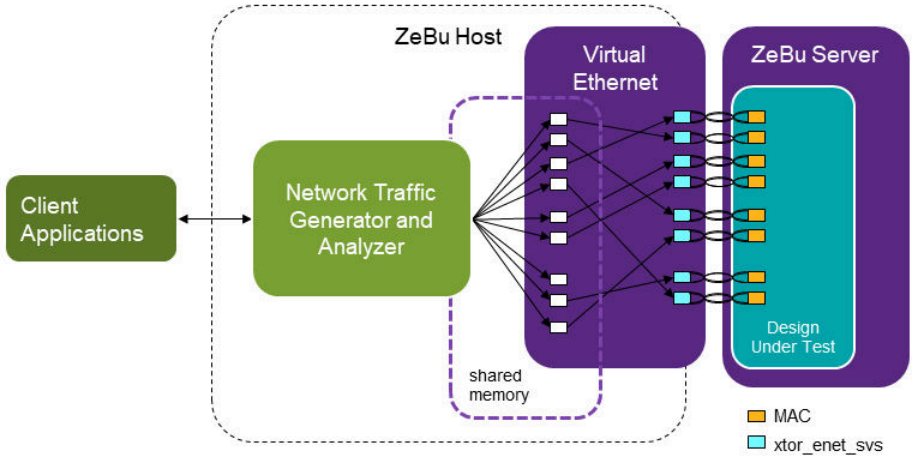
See the following topics for more information:

Overview

The networking validation solution leverages the power of virtual test platforms to provide a more efficient and cost-effective way to perform pre-silicon validation. By combining Synopsys ZeBu emulator, Synopsys ZeBu Ethernet transactor, and third party network traffic generator and analyzer, such as Spirent TestCenter or Keysight IxVerify, the solution enables customers to perform comprehensive pre-silicon validation in a virtual environment.

Feedback

Figure 1 Overview of Virtual Ethernet



Benefits of Virtual Ethernet

The solution offers several key benefits for customers, including:

- *Faster time-to-market*: By using virtual test platforms to perform pre-silicon validation, the solution can significantly reduce the time required to bring a networking product to market. This is because virtual test platforms enable customers to perform testing and validation in parallel with other development activities rather than waiting for physical prototypes to be manufactured.
- *Lower costs*: Traditional pre-silicon validation methods can be very expensive because of the need for physical prototypes and specialized testing equipment. However, the solution uses virtual test platforms that are cost-effective and be easily scaled to meet the needs of any project.
- *Higher quality products*: By performing comprehensive pre-silicon validation using the solution, customers can identify and fix issues earlier in the development process, before they become more costly to fix. This results in higher quality products that are more reliable and have fewer defects.

Compatibility With Spirent TestCenter and Keysight ixVerify

The solution is compatible with both *Spirent TestCenter* and *Keysight IxVerify* virtual test platforms, giving customers the flexibility to choose the platform that best meets their needs. Both platforms are market leaders in system validation for networking solutions and offer a wide range of features and capabilities to support pre-silicon validation.

2

Host Reconfiguration

This solution can be deployed to most Linux servers available after few system reconfigurations. There reconfigurations are done for the following three catalog:

- Virtualization Setup
- Network Configuration Setup
- Hugepage Setup

The setup has been validated on the following operating systems:

- CentOS 7.3
- CentOS 7.9
- AlmaLinux 8.4
- SUSE 12.5

Alternatively, ZeBu Virtual Ethernet offers a script based on the CentOS and Red Hat Enterprise Linux (RHEL) operating systems, designed to help you reconfigure the ZeBu host according to the requirements discussed in sections listed. It is important to be aware that the script might need to be adapted to conform to the specific security policies and target operating systems of each site. To know more about the script, refer to Automated Reconfiguration Script.

Virtualization Setup

To reconfigure the virtualization setup, perform the following steps:

Note:

The following configurations are mandatory and in most cases root access is required.

1. Enable VT-x (Virtualization Technology) and VT-d (Virtualization Technology for Directed I/O) in BIOS setting.
2. Disable SELinux by using the following setting in the file */etc/selinux/config*:

   ```
   SELINUX=disable
   $ sudo sed -i -e "s/SELINUX=.*/SELINUX=disabled/g" /etc/selinux/config
   ```

3. Install KVM and related virtualization packages. Note that the steps may differ depending on the Linux distribution.

   For CentOS/RHEL 7.x, install these packages with *yum*:

   ```
   $ sudo yum install qemu-kvm virt-manager libvirt libvirt-install
    libguestfs-tools
   ```

   For AlmaLinux 8, install these packages with *dnf*:

   ```
   $sudo dnf install qemu-kvm virt-manager libvirt libvirt-client
    libguestfs-tools
   ```

   For SUSE Linux OS 12.5, install these packages with *zypper*:

   ```
   $sudo zypper install qemu-kvm virt-manager libvirt libvirt-client
    libguestfs-tools
   ```

4. For Ubuntu 20.04, install these packages with apt:

   ```
   $ sudo apt install qemu-kvm virt-manager libvirt-daemon-system
    libvirt-clents bridge-util
   ```

   Before proceeding with the installation of KVM and related virtualization packages, it is advisable to consult with your IT department regarding steps to follow for your Linux distribution. They might have their own guidelines or protocols that need to be followed for the installation.

Modification to the libvirt.conf File

In this solution, *libvirt* is used as a means to deploy virtual machines from third-party traffic generators and analyzers. In order for non-root users to be able to launch virtual machines without root permission, modify the *libvirtd.conf* file. This is available at the following location */etc/libvirt/libvirtd.conf*

Feedback

To modify the *libvirt.conf* file, perform the following steps:

1. Login as root, modify the file */etc/libvirt/libvirtd.conf*:

   a. Uncomment the lines around *unix_sock**. It is required so users belonging to certain Linux group can launch virtual machines with *virsh* or *virt-manager*:

   ```
   unix_sock_group = "libvirt"
   unix_sock_rw_perms = "0770"
   ```

   b. Uncomment the line *auth_unix_rw="none"*. It is required because network communication between virtual machines and the host system are implemented through Unix domain socket:

   ```
   # Set an authentication scheme for UNIX read-write sockets
     # By default socket permissions only allow root. If PolicyKit
     # support was compiled into libvirt, the default will be to
     # use 'polkit' auth.
     #
     # If the unix_sock_rw_perms are changed you may wish to enable
     # an authentication mechanism here
     auth_unix_rw = "none"
   ```

2. If libvirtd is not yet started:

   ```
   $ sudo systemctl start libvirtd
   ```

3. If libvirtd has already been started, restart it so the changes in libvirtd.conf become effective:

   ```
   $ sudo systemctl restart libvirtd
   ```

User Management

After libvirtd is reconfigured and restarted, system administrator needs to add users to the Unix group previously specified as unix_sock_group in /etc/libvirt/libvirtd.conf. By doing so, users from that Unix group are granted permission to access the virtualization features of the system.

For example, to add users to the *libvirt* group, use the following command:

```
$ sudo usermod -aG libvirt <username>
```

Network Configuration Setup

The following paragraph introduces the common networking configurations used by this solution. For more details, see https://wiki.libvirt.org/Networking.html

The two common setups are as follows:

- Shared Physical Devices (or Bridged Networking): Requires distribution-specific manual configuration.
- Virtual Networks (NAT Forwarding): Is identical across all distributions and available out-of-the-box.

Shared Physical Devices (or Bridged Networking)

The NAT-based connectivity adopted in virtual network is useful for quick and easy deployments, or on machines with dynamic or sporadic networking connectivity. For users who want to connect to virtual machines from desktop applications running remotely, full bridging is required.

The instructions for setting up bridged networking vary by distributions, and even by releases. Therefore, consult your IT department or Synopsys AE team for assistance for the exact commands.

The steps to set up bridged networking in a host running CentOS 7.3 are as follows:

1. Check if bridged networking is already setup:

   ```
   $ sudo brctl show
   ```

   This command shows a list of bridges on the system along with their interfaces as follows:

   ```
   bridge name         bridge id           STP enabled       interfaces
           br0                 8000.ac1f6b0c3958   no
    eno1
           br1                 8000.000000000000   no
           virbr0              8000.5254005731ca   yes
    virbr0-nic

    vnet0
   ```

   If there is at least one bridge listed and one of the interfaces is *eth0* or *eno1*, then the machine is likely configured as a "*bridged network*". If this is the case, you can skip the configuration steps outlined in this section and proceed to the next section on Virtual Networks (NAT Forwarding).

   Note:

   brctl is part of the Linux package bridge-utils.

2. Identify the name of the network interface on your host:

   ```
   $ /sbin/ip route
   default via 10.195.7.254 dev eth0
   10.195.0.0/21 dev eth0 proto kernel scope link src 10.195.0.190
   ```

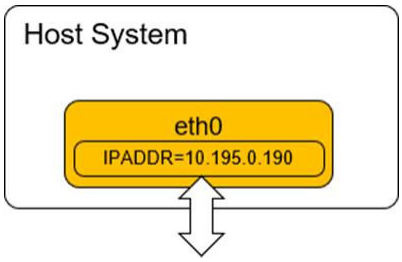
Feedback

3. Based on the output received, the original networking device is *eth0*.
4. Find out the ip address, gateway, netmask setting of *eth0*:

```
$ /sbin/ip addr show eth0
10: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue
 state UP qlen 1000 link/ether ac:1f:6b:0c:39:58 brd ff:ff:ff:ff:ff:ff
inet 10.195.0.190/21 brd 10.195.7.255 scope global br0 valid_lft
 forever preferred_lft forever
inet6 fe80::ae1f:6bff:fe0c:3958/64 scope link valid_lft forever
 preferred_lft forever
```

5. Based on the output of *ip addr*, the IP address of the host is *10.195.0.190*.

Figure 2 IP address of the host system



6. Check if dhcp is used:

```
$ grep BOOTPROTO /etc/sysconfig/network-scripts/ifcfg-eth0 |grep -i
 dhcp
```

7. Create a new file */etc/sysconfig/network-scripts/ifcfg-br0*:
   - If dhcp is used, set *BOOTPROTO* to *dhcp*:

```
DEVICE=br0
TYPE=Bridge
ONBOOT=yes
BOOTPROTO=dhcp
NM_CONTROLLED=no
STP=off
IPV6INIT=no
```

   - If static IP is used, set `BOOTPROTO` to `none`. Also specify `IPADDR`, `` `GATEWAY` `` and `NETMASK` based on the value from the file:/etc/sysconfig/network-scripts/ifcfg-eth0

```
DEVICE=br0
TYPE=Bridge
ONBOOT=yes
BOOTPROTO=none
NM_CONTROLLED=no
STP=off
IPV6INIT=no
IPADDR="10.195.0.190"
```

Feedback

```
GATEWAY="10.195.7.254"
NETMASK="255.255.248.0"
```

8. Modify the file /etc/sysconfig/network-scripts/ifcfg-eth0:

   a. Comment out the line with *IPADDR*, *GATEWAY* and *NETMASK* specified:

```
$ sed -pi \
-e "s/IPADDR/#IPADDR//g" \
-e "s/GATEWAY/#GATEWAY//g" \
-e "s/NETMASK/#NETMASK//g" \
/etc/sysconfig/network-scripts/ifcfg-eth0
```

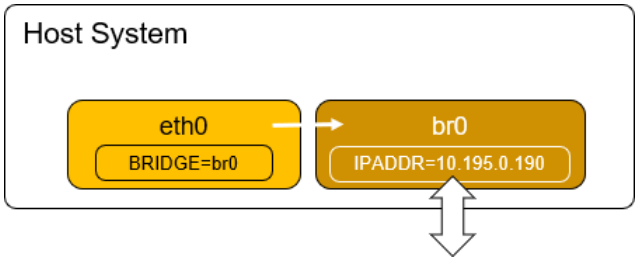
9. Add a new line with `BRIDGE=br0`:

```
$ echo "BRIDGE=br0" >> /etc/sysconfig/network-scripts/ifcfg-eth0
```

10. Reboot the host.

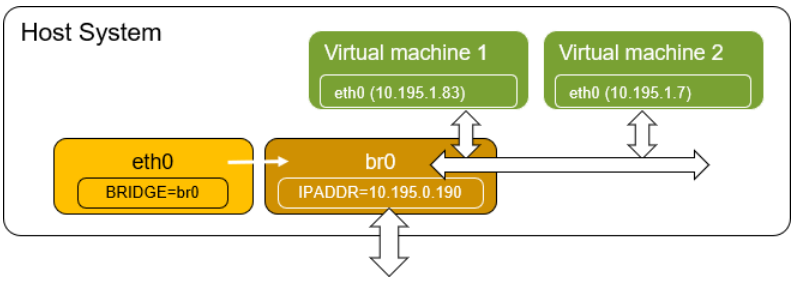
    After the host is rebooted, the new networking topology looks like the following:

Figure 3 Network Topology After the Host is Rebooted



User can now start new virtual machines with their networking devices connected to `br0`. The virtual machines should be accessible after they are assigned with valid static IP address as shown in the following figure:

Figure 4 Networking Devices Connected to br0



Virtual Networks (NAT Forwarding)

1. By default, `libvirt` creates a virtual network named default. Check if it is available:

```
$ sudo virsh net-list
Name         State    Autostart   Persistent
default      active   yes         yes
```

2. If it is not defined or active, define and then start it:

```
$ sudo virsh net-define /etc/libvirt/qemu/networks/default.xml
$ sudo virsh net-start default
$ sudo virsh net-autostart default
```

3. The virtual network has the virtual bridge `virbr0`, which has an IP address of `192.168.122.1`. The bridge also provides DHCP services with the following parameters:

   - range: `192.168.122.2-192.168.122.254`

     Any virtual machine connected to the virtual bridge, `virbr0`, can be assigned an IP address through DHCP in the specified range. Note that the virtual machine can be accessed over its IP address from the local host system:

     ```
     $ ssh admin@192.168.122.4
     ```

4. Users might want to modify the DHCP range (with `virsh`). The following example shows how to change the DHCP range to `192.168.122.128` to `192.168.122.254`:

```
$ sudo virsh net-edit default
<network>
<name>default</name>
<uuid>eddd03ff-5825-42ef-bc99-968bddf773c2</uuid>
<bridge name='virbr0' stp='on' delay='0'/>
<mac address='52:54:00:67:75:b1'/>
<ip address='192.168.122.1' netmask='255.255.255.0'>
<dhcp>
<range start='192.168.122.128' end='192.168.64.254'/>
</dhcp>
</ip>
</network>
```

5. Save the file to apply changes.

   For more details on the Network XML format, see https://libvirt.org/formatnetwork.html.

Modification to the bridge.conf file

By default, QEMU-KVM requires root privileges to start a virtual machine with a network bridge interface. This is because the network bridge interface requires privileged access to the host network stack.

Adding the `allow br0` and `allow virbr0` lines to the bridge.conf file at /etc/qemu-kvm/bridge.conf specifies that non-root users can use the virtual bridge interface, which allows virtual machines to connect to the host network. This lets non-root users to start virtual machines with network connectivity without requiring root privileges.

Users might also add the line `allow all` to the bridge.conf file. This lets non-root users to use any virtual bridge interface in the system.

Hugepage Setup

ZeBu Virtual Ethernet lets data to be exchanged between ZeBu transactors and virtual machines that generate and analyze network traffic with shared memory. It is a process in which a lot of memory access takes place, and one way to improve performance is to use hugepage. The bigger the hugepage, the more efficient is the data access.

Most Linux systems support 2 sizes of hugepage: *2MB* and *1GB*. The default size is *2MB*, and they can get allocated at runtime. However, if a lot of hugepages are needed or if the hugepage size is 1GB, they should be reserved at boot time or the system might not be able to allocate them due to memory fragmentation.

If performance is critical, you should reconfigure the host to run with *1GB* hugepage instead of the default *2MB* hugepage. To check if hugepages are already setup, perform the following steps:

```
$ grep ^Huge /proc/meminfo
HugePages_Total:0
HugePages_Free:0
HugePages_Rsvd:0
HugePages_Surp:0
Hugepagesize:2048 kB
```

If the right *Hugepagesize* and *HugePages_Rsvd* are shown, proceed to Creating a Hugepage Mount Point.

Feedback

Specifying Hugepage Size and Reserving Hugepages

To specify hugepage size and reserve enough hugepages at boot time, perform the following steps:

1. Login as a root user.
2. Add or modify `GRUB_CMDLINE_LINUX_DEFAULT` setting in the /etc/default/grub configuration file for the `GRUB/GRUB2` bootloader on a Linux system. Here are some examples:
   - To setup `1GB` hugepage and reserve 32 pages at boot time:

     ```
     GRUB_CMDLINE_LINUX_DEFAULT="quiet hugepagesz=1G hugepages=32
      apparmor=0"
     ```

   - To setup `2MB` hugepage and reserve 4096 pages at boot time:

     ```
     GRUB_CMDLINE_LINUX_DEFAULT="quiet hugepagesz=2M hugepages=4096
      apparmor=0"
     ```

   - **Note:**

     The specific options and values used in the `GRUB_CMDLINE_LINUX_DEFAULT` setting might vary depending on the system and its configuration.
3. Reboot the host machine.

Creating a Hugepage Mount Point

The last step is to create a mount point of type `hugetlbfs` with the right permission so that the authorized users might access the hugepages.

- To mount 32 pages of 1GB hugepage at /mnt/huge, for the UNIX group `libvirt` with a group ID of 31, do the following:

  ```
  $ sudo mkdir -p /mnt/huge
  $ sudo mount -t hugetlbfs nodev /mnt/huge -o
   pagesize=1G,size=32G,mode=775,gid=31
  $ sudo chgrp libvirt /mnt/huge
  $ sudo chmod 775 /mnt/huge
  ```

-
- To mount 4096 pages of 2MB hugepage at /mnt/huge, replace with `pagesize=2M,size=8192M`.

  The choice of `unix_group` should match the setting of `unix_sock_group`specified in /etc/libvirt/libvirtd.conf.

Feedback

- Alternatively, use the standard Linux tool `hugeadm`, if available. For example:

```
$ sudo hugeadm --create-group-mounts=<unix_group>
$ sudo chmod -R 775 /var/lib/hugetlbfs/group/<unix_group>
$ sudo chgrp -R <unix_group> /var/lib/hugetlbfs/group/<unix_group>
$ sudo hugeadm --pool-pages-min 1GB:32
$ sudo hugeadm --pool-pages-max 1GB:32
```

Automated Reconfiguration Script

ZeBu Virtual Ethernet offers a script based on the CentOS and RHEL operating systems. It is designed to help you reconfigure the ZeBu host according to the requirements discussed in sections listed. It is important to be aware that the script might need to be adapted to conform to the specific security policies and target operating systems of each site.

To execute the script, login as root and perform the following steps:

```
$ $ZEBU_VTG_ROOT/scripts/install/setup-host.sh
```

The script contains the following four sections:

- KVM Health Check
- Network Bridge Setup
- Hugepage Configuration
- Write Out

KVM Health Check

This section checks if the host meets the minimum system requirements and prepare scripts to enable KVM. The installation exits prematurely if the system fails the health check. No user input is required in this section.

Network Bridge Setup

This section prepares scripts to create the virtual public network bridge, br0, and reconnect network devices after the bridge is created. No user input is required in this section.

Feedback

Hugepage Configuration

This section prepares scripts and configuration files to reserve hugepages for ZeBu Virtual Ethernet use. Since hugegpage configuration is somewhat site specific, users are prompted with the following questions:

1. How many pages of 1G hugepage are to be allocated? The minimum number of hugepage required is 1GB per CPU socket per ZeBu session. If there are 2 CPU sockets, a minimum of 2 pages should be allocated. If 1G hugepage is not available, a minimum of 2 x 1G / 2M = 2048 of 2MB hugepage should be allocated.

   Users might determine the number of CPU socket with the following command:

   ```
   $ lscpu |grep Socket
   ```

2. Specify the Unix group for the hugepages created [Hit enter to skip]? Specify the Unix group that would have access to the hugepages reserved.
3. Specify the mount point to the hugepage memory [Default: /dev/hugepages] : The hugepage memory allocated is mounted to the file system so ZeBu Virtual Ethernet may access it.

Write Out

This section prepares a TAR file with all scripts and configuration files generated from the previous sections. No input is required in this section. The command you may execute to apply the changes is displayed at the end of this section.

After reviewing the content in the TAR file with your system administrator, apply the changes to the host and reboot the machine. As there may be site specific policy on system reconfiguration, it is important to get system administrator involved in this process.

3

Install and Setup

The hardware and software requirements, licenses and the steps to install the Virtual Ethernet package are mentioned in the common VSA installation guide. For details, refer to the *ZeBu Virtual System Adaptors Installation Guide*.

After successful installation of the Virtual Ethernet package, you need to perform the following steps:

- Review the Directory Structure
- Install QEMU
- Install Keysight IxVerify
- Install Spirent TestCenter Virtual
- Setting Environment Variables

Review the Directory Structure

The following directory structure is created after installing Virtual Ethernet:

```
├── api ......................... Virtual Ethernet API
│   ├── protos
│   ├── python
│   │   └── vtg
│   └── tcl
│       └── vtg1.0 .............. Currently supported API: Tcl API vtg1.0
│
├── bin ......................... PCAP Utilities
├── doc ......................... Documentation
│   ├── foss
│   └── man ..................... Root of manual page hierarchy
├── example
│   ├── ex1-zebu ................ Example shows how to integrate Virtual
 Ethernet
│   │                            with ZeBu as C++ testbench
│   ├── ex2-zrci ................ Example shows how to integrate Virtual
 Ethernet
│   │                            with ZeBu as a shared object loadable
 to zRci
```

Feedback

```
│   ├── ngtb-example .............Example to integrate Virtual Ethernet
using ngtb
│   ├── ex-sem    ................Example to compile a dynamic shared
library
│   │                            for zRCI with SEM support
│   ├── model.mii ................ HDL Source and scripts to build
│   │   │                        ZeBu model with xtor_enet_svs over MII
│   │   ├── common
│   │   ├── sem
│   │   └── zebu
│   ├── model.pcs ................ HDL Source and scripts to build
│   │   │                        ZeBu model with xtor_enet_svs over PCS
│   │   ├── common
│   │   ├── sem
│   │   └── zebu
│   ├── README.md ................ Description of HDL testbench, model
build
│   │                            and C++ testbench build process
│   ├── tests .................... Examples show how user may interact
with
│   │   │                        Virtual Ethernet
│   │   ├── tcl
│   │   └── python
│   │   ├── etc
│   └── utils .................... Example shows how user may hook up
their
│       │                        own program with Virtual Ethernet
│       ├── pcs_monitor.bin
│       └── register_cb_pcs
.
.
.
├── include ..................... Include files
├── lib ......................... Shared libraries common to CXXABI=0/1
│   └── vhost                    (CPU micro-architecture specific libs)
│       ├── default ............. Core2
│       ├── hsw ................. Haswell / Broadwell
│       ├── knl ................. Knight Landing / Skylake
│       └── snb ................. Sandy Bridge / Ivy Bridge
├── libABI0 ..................... Shared libraries, built with CXXABI=0
│   ├── enetxtor
│   │   └── Q-2021.06-SP3 ....... Shared libraries when xtor_enet_svs
from
│   │                            Q-2021.06-SP3 is used
│   └── vhost -> ../lib/vhost
├── libABI1 ..................... Shared libraries, built with CXXABI=1
│   ├── enetxtor
│   │   └── S-2023.03 ........... Shared libraries when xtor_enet_svs
from
│   │                            S-2023.03 is used
│   └── vhost -> ../lib/vhost
└── scripts
    ├── install .................. Scripts used during installation
```

Feedback

```
    │   ├── libvirt
    │   ├── qemu
    │   ├── README ............... Description of steps to complete
installation
    └── runtime .................. Scripts used at run time
```

Install QEMU

To run virtual machines from third-party network traffic generators and analyzers, QEMU is used as the hypervisor. The version of QEMU used in Virtual Ethernet has been slightly modified from Version 4.1.1. The steps to install QEMU are as follows:

1. Login as administrator and execute the following script:

   ```
   $ $ZEBU_VTG_ROOT/scripts/install/setup-qemu.sh
   ```

2. Specify the destination directory when you are prompted for it.
3. After the QEMU installation, set the environment variable `ZEBU_VTG_QEMU` to the destination directory that you have specified.
4. Check your bridge-helper permissions.

   ```
   sudo chmod u+s /usr/lib/qemu-bridge-helper
   ```

5. Change and check your vhost-net file permissions.

   ```
   sudo chmod 777 /dev/vhost-net

   ls -l /dev/vhost-net
   ```

Install Keysight IxVerify

Keysight IxVerify is a design verification software that validates electronic designs before manufacturing. It provides prebuilt test suites, integration with design environments, supports simulation and emulation modes, and includes a user-friendly interface and advanced debugging capabilities. It ensures designs are reliable, efficient, and meet intended functionality.

The IxVerify solution requires two types of virtual machines (VMs) to be deployed: the virtual load module (VLM) and the virtual chassis.

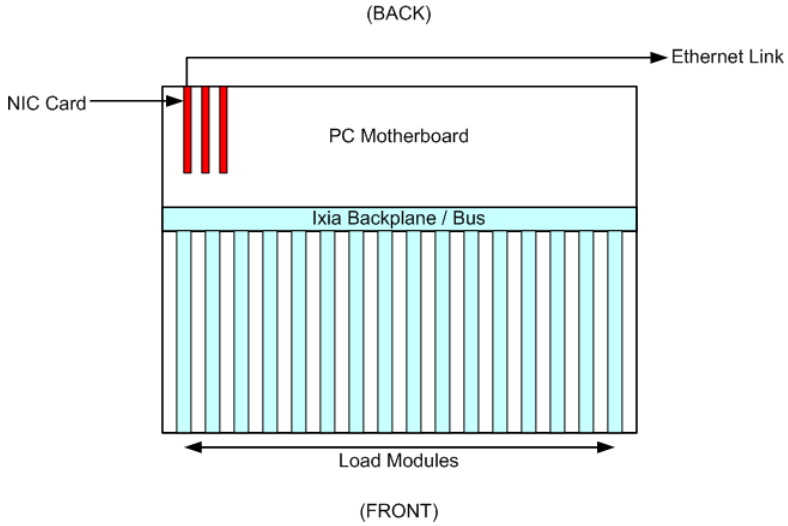
Virtual Load Module

In Keysight terminology, a "card" refers to a hardware module that can be inserted into a chassis to provide additional network ports and processing capabilities. A chassis typically contains several card slots, and each card can be configured with a different set

Feedback

of network interfaces. A VLM is a software-based emulation of the Ixia hardware card, which allows users to simulate network traffic and test network equipment without the need for physical hardware. It virtualizes the "card" by running it as a virtual machine on a hypervisor, such as VMware or KVM.

Figure 5 Drawing of Physical Ixia Chassis and Load Module



Each VLM supports up to 25 ports. Each port is connected to a `xtor_enet_svs` transactor in the ZeBu model. The transactor is responsible for delivering the network traffic produced by a port in the virtual machine to a MAC in a design under test.

Virtual Chassis

The virtual chassis is a software-based emulation of the Ixia hardware chassis, which provides the infrastructure for managing and controlling the virtual load modules. The virtual chassis is hosted on a hypervisor, such as VMware or KVM, and can be accessed and configured using a management interface, such as IxNetwork and IxOS Tcl Client.

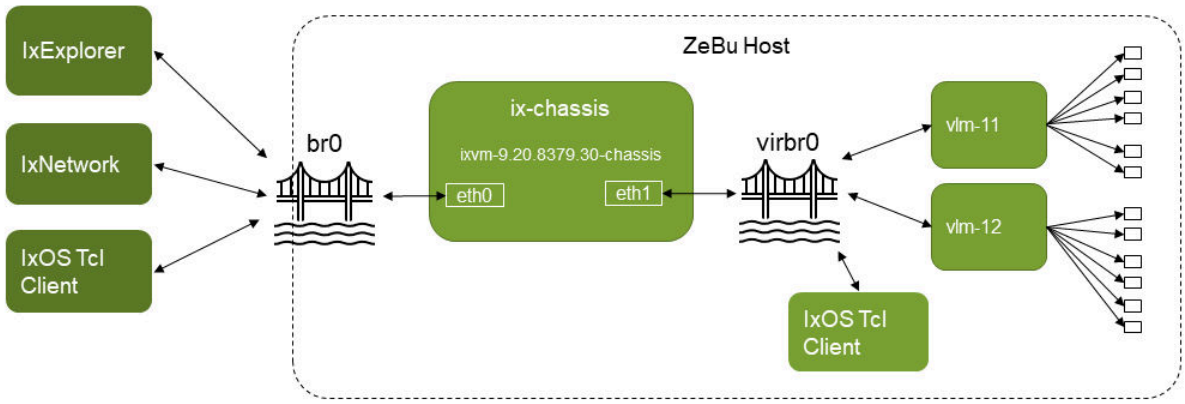
This virtual machine has 2 network devices: a management port `(eth0)` which is connected to physical networks using network bridges, and a test port `(eth1)` which is connected to a number of virtual load modules over a virtual network bridge.

It is typical that the management port of the chassis is assigned with a static IP address, so it can be identified from applications running remotely. A virtual chassis can manage up to 32 virtual load modules.

Feedback

Network Connection Between VMs

Figure 6 Virtual Machines Network Connections



One of the host requirements of this solution is the creation of the virtual network bridge `br0`. By connecting the management port of the chassis to `br0`, the chassis becomes accessible from the physical networks as if it was a physical machine.

On the other hand, the chassis communicates with its virtual load modules over the virtual bridge`virbr0`, which is part of the virtual network "default" created and managed by libvirt.

Just to recap, the "default" virtual network created by libvirt has the following characteristics:

- it is connected to the host system using a virtual network interface called virbr0.
- it is assigned the IP address `192.168.122.0/24`
- it is configured to provide DHCP services to virtual machines connected to it.
- it is configured to use NAT networking, meaning that virtual machines connected to it can communicate with each other and with the host system, but they are not visible on the external network.

When a chassis is started, its test port is connected to `virbr0`, which assigns an IP address, in the range of `192.168.122.2-192.168.122.254`, to the test port of the chassis. Similarly, virtual load modules connected to the same `virbr0` is assigned an IP address in the same range. As a result, network communication between chassis and virtual load modules is achieved.

If user want to start another chassis in the same ZeBu host, just create a new virtual network with a different virtual network interface and assign a different static IP address of the management port of the chassis.

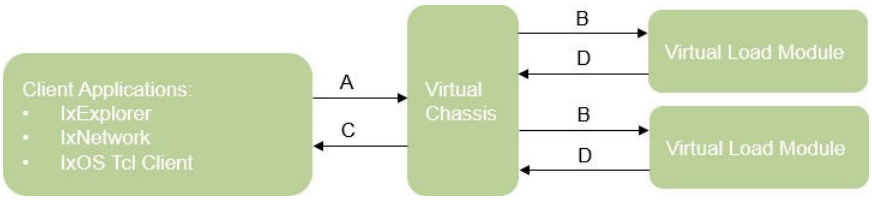
Feedback

Additional information about IxVerify deployment can be found in the “IxVerify – Installation over KVM over CentOS” document, which is available from the Ixia website.

TCP/UDP Requirements

IxVerify Client Applications (IxExplorer, IxNetwork, and IxOS Tcl Client) communicate with the Virtual Chassis over a number of TCP and UDP ports.

Figure 7 TCP and UDP ports



As per Chapter 5 "TCP/UDP ports" of "IxVerify Reference Guide", the following TCP and UDP ports are required:

- Connection A (Client Applications to Virtual Chassis):
  - TCP Ports: `22, 80, 443, 111, 2601, 998-999, 1000, 1080, 2345, 3222, 3601, 4501-4502, 4601, 5285-5286, 5326-5327, 5480, 5488-5489, 6001-6005, 6665, 6967, 6978, 8021, 8022, 8881, 8989-8890, 9101-9102, 9613-9676, 10115, 10116, 10119, 17662, 17668-17777, 18765, 23123, 21653`
  - UDP Ports: `67, 68, 123, 161, 162, 605, 1000, 6004, 10116`
  - ICMP
- Connection B (Virtual Chassis to Virtual Load Module):
  - TCP Ports: `22, 25, 110, 111, 143, 443, 998-999, 1000, 1001, 1281, 1283, 2121, 2205, 2209, 2601, 3601, 3222, 4119, 4601, 6001, 6665, 6690 -6692, 6857, 6901, 6929, 7768, 7769, 8008, 8022, 8881, 8989, 9020, 9101, 9102, 9613-9616, 10116, 14425, 16566, 16570, 16574, 16665, 16666,17674, 18408`
  - UDP Ports: `67, 68, 123, 161, 162, 666, 1000, 10116`
  - ICMP

- Connection C (Virtual Chassis to Client Applications):
  - ICMP
- Connection D (Virtual Load Module to Virtual Chassis):
  - ICMP

Request firewall access to the TCP/UDP ports above for IxVerify to function properly.

Note:

Note that the exact TCP/UDP ports may change from one IxVerify release to another. Consult Keysight representative for the exact port number.

Required Keysight IxVerify Software

- Virtual Ethernet supports IxOS 9.20.8739.30 or later. Contact Keysight Support to download the following images and installers:
  - `Ixia_Virtual_Load_Module_9.20_KVM.qcow2.tar.bz2`
  - `Ixia_Virtual_Chassis_9.20_KVM.qcow2.tar.bz2`
  - `IxOS9.20.23.2Linux64.bin (IxOS Tcl Client)`
- For IxExplorer users, download the following:
  - `IxExplorer9.20.exe (MS Windows installer)`
- For IxNetwork users, download the following:
  - `IxNetwork9.20.exe (MS Windows installer)`
  - `IxNetworkWeb_KVM_9.20.2112.52.qcow2.tar.bz2 (web edition)`

The steps to install IxOS, IxNetwork and IxExplorer are not covered in this document. Contact Keysight for assistance.

Keysight License Server

The Keysight license server is a software tool that allows multiple users to share licenses for Keysight products across a network. It acts as a central repository for license files and manages the allocation of licenses to users who request them. To use Keysight IxVerify, users must first configure their Linux hosts for the Virtual Chassis to connect to the license server and request licenses as needed.

Note that the steps to setup Keysight license server is not covered in this document. Contact Keysight representative for details.

Launching Virtual Load Modules

Virtual Ethernet provides a framework to start virtual load modules in an automatic fashion. This framework involves:

- User maintaining their own copies of qcow2 image for Virtual load modules
- The virtual network "default" and the virtual bridge virbr0, created and managed by libvirt, are up and running
- libvirt is setup to allow non-root users to launch virtual machines
- User specifying the following in a Virtual Ethernet YAML configuration for the following:
  - number of virtual load modules to be launched
  - number of virtual ports available in each virtual load module
  - qcow2 image location of these virtual load modules
  - One of the following set of information:
    - `eth, virbr`

      name of virtual bridge to be used (if it is different from virbr0) IP address of the chassis management port and test port
    - `domain`

      domain name of the Virtual Chassis

The steps to setup libvirt is discussed in Host Reconfiguration and the parameters to specify in Virtual Ethernet YAML configuration is covered in Configure YAML Fields.

Launching Virtual Chassis

While the launching of Virtual load module is always user-specific, the launching of a Virtual Chassis is more organization specific.

Virtual Chassis as System Level Service

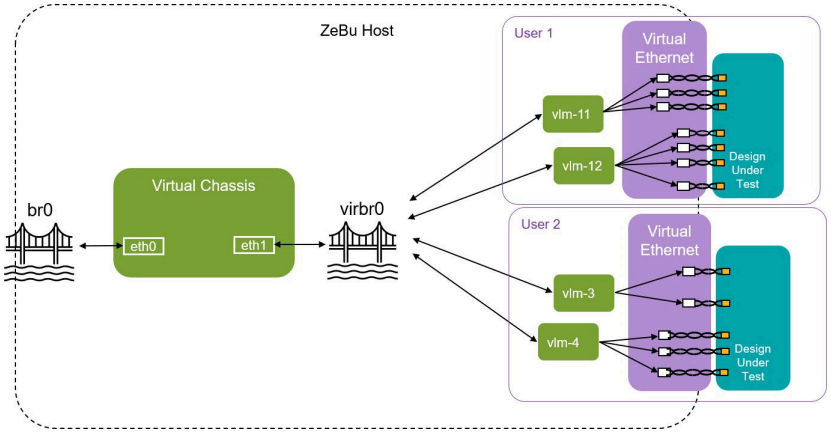
Some organizations prefer to set up a Virtual chassis that is shared among users. In this case, the Chassis is created as a system-level virtual machine (that is, it is connected to qemu:///system in QEMU scheme, and is typically started, stopped, or accessed with

Feedback

sudo) and is managed by a system administrator from the organization. Regular users are not allowed to login or reconfigure the Chassis. This scheme works well if:

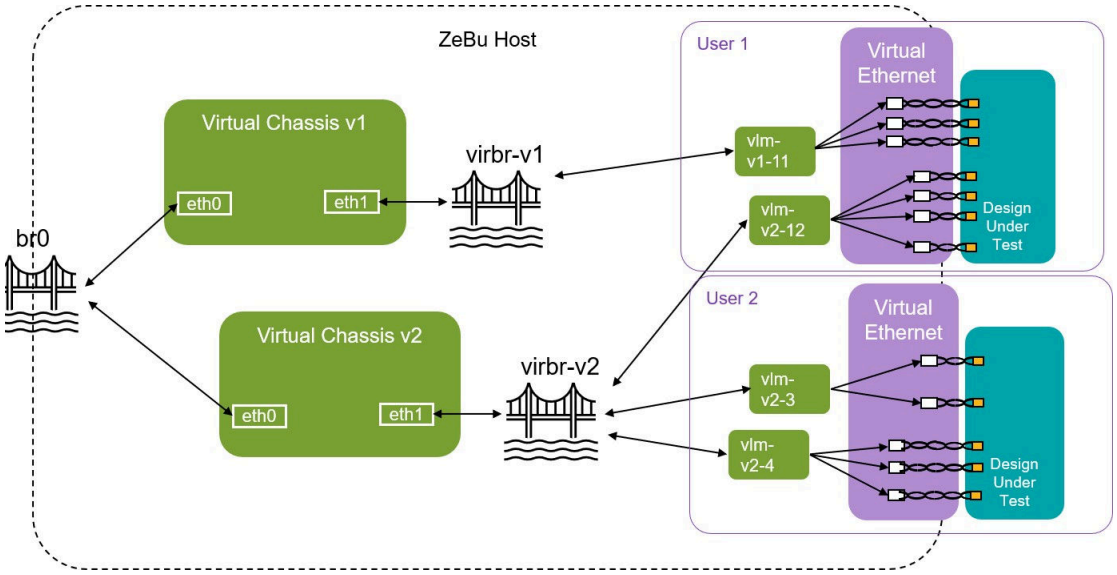
- All users use the same version of IxVerify.
- Users are coordinated so each of them are connecting their virtual load modules as different cards of the Virtual Chassis.

Figure 8 Virtual Chassis as System Level Service



If users need different version of Virtual chassis, the topology needs to be updated. One way to support multiple version Virtual Chassis is to duplicate the setup with a new virtual bridge. Users decide which virtual chassis they should connect based on the version of virtual load modules they have launched:

Figure 9 Multiple Version Virtual Chassis



Feedback

The advantages of this scheme are:

- Only system administrator needs to understand how to setup the Chassis.

The drawbacks of this scheme are:

- Virtual Chassis are virtual machines and they consume CPU resources.
- It is less flexible because:
  - If users need an upgrade of Chassis version, they need to get system administrator involved.
  - If users fail to connect to the Chassis, or their virtual load modules do not get brought up properly, it is very hard to debug if users have no access to the console of the Chassis.

Virtual Chassis as User Level Process

On the other hand, some organizations prefer the Virtual Chassis to be managed by the users for greater flexibility. Under this scheme, the qcow2 image of the virtual machine is maintained by the user, and the virtual machine is launched by users in qemu:///session. Only the owner of the virtual machine can access it.

This is the scheme prompted by Virtual Ethernet, with scripts provided as part of the package to automate the launching of Virtual Chassis.

Virtual Chassis Launcher Script

Note:
Ensure that the ZeBu host is already reconfigured as per the description in Host Reconfiguration

Take a look at the file $ZEBU_VTG_ROOT/example/tests/ixtcl/client.tcl. A few Tcl variables are defined at the top of this file:

```
# pre-define eth0 setting of supported zebu host
 set hostmap(amdepyc2)    {10.194.136.80 255.255.248.0 10.194.143.254}
 set hostmap(vgzeburt70)  {10.195.1.102  255.255.248.0 10.195.7.254}

 # default setting for chassis defined in qemu:///session
 set ixvm_qcow2
"/u/vtg/9.20.8739.30/Ixia_Virtual_Chassis_9.20_KVM.qcow2"
 set ixvm_domain "ixvm-9.20.8739.30-chassis"
 set ixvm_eth0_br "br0"
 set ixvm_eth1_br "virbr0"
 set license_server 10.15.214.10
 set ixvm_password "admin"
```

Here is the description of each parameter:

Table 1 Virtual Chassis Parameters

Parameter	Description
ixvm_qcow2	The path to a Virtual Chassis qcow2 image.
ixvm_domain	The name of the virtual machine instance running in a libvirt.
ixvm_eth0_br	The network bridge to be connected to the management port.
ixvm_eth1_br	The network bridge to be connected to the test port.
license_server	The host name or IP address of the license server for IxVerify.
ixvm_password	The administrator password (login: admin) of the Chassis. User may change the administrator's password via the Chassis console. If user does change.

User might execute this Chassis launching script with the following command:

```
$tclsh $ZEBU_VTG_ROOT/example/tests/ixtcl/client.tcl
```

Note:
SSHpass is required for this Chassis launching script to work. For details on how to install SSHpass on the host, see sshpass

Here is an example of the output when the Chassis is successfully launched.

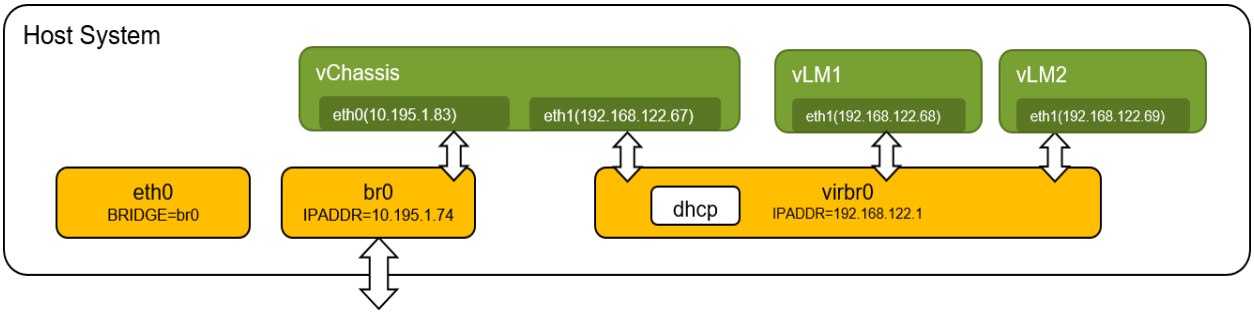
```
##### Check if br0 is available ...
##### Check if virbr0 is available ...
##### Check if ixvm-9.20.8739.30-chassis is running ...
##### - Define or Redefine ixvm-9.20.8739.30-chassis ...
##### - Start ixvm-9.20.8739.30-chassis ...
##### Find out IPv4 address of Chassis's interfaces ...
##### - eth0: 10.195.1.107 dev br0  src 10.195.2.1
##### - eth1: 192.168.122.128 dev virbr0  src 192.168.122.1
##### Remove keys belonging to 192.168.122.128 from '~/.ssh/known_hosts'
##### Run 'set ixverify-sync-server' over eth1 ...
##### Setup Ixia Tcl Environments ...
Tcl Client is running Ixia Software version: 9.20.23.2
##### Check if eth0 needs to be reconfigured ...
##### Run 'set license-server' over eth1 ...
Connecting to Tcl Server 192.168.122.128 ...
9.20.8739.30
Connecting to Chassis 1: 192.168.122.128 ...
```

Feedback

First Time Chassis Setup

When a chassis is started, its test port,*eth1*, is connected to *virbr0*and is assigned an IP address in the range of `192.168.122.2-192.168.122.254` by the DHCP service in the virtual bridge. Similarly, virtual load modules connected to the same*virbr0* is assigned an IP address in the same range. As a result, network communication between virtual chassis and virtual load modules is achieved. Figure 10 shows one of the possible networking configurations after IP addresses are assigned by DHCP.

Figure 10 Chassis Setup



Note that this scheme works only if test port of the Virtual Chassis is setup to use DHCP by default. However, it may not be the case: the qcow2 image provided by Keysight does not necessary have their network devices set to use DHCP. Therefore, the virtual machine may need to be setup manually for the first time.

The chassis launching script reports an error or does not respond, if the test port in the Chassis does not have the correct setup:

```
##### - eth1: not found
Please setup eth1 (either static IP address or dhcp) via virsh console
 first!
    while executing
"error "Please setup eth1 (either static IP address or dhcp) via virsh
 console first!""
    invoked from within
"if {$ixvm_eth1_ip == ""} {
  error "Please setup eth1 (either static IP address or dhcp) via virsh
 console first!"
}"
    (file "ixtcl/client.tcl" line 77)
```

Feedback

When it happens, follows these steps to reconfigure the test port of the Chassis:

1. Connect to the console of the Virtual Chassis:

```
$ virsh console <ixvm_domain>
```

2. Login as an administrator (login name: admin, password: admin). Enter the following commands over the CLI of the Virtual Chassis:

```
# set ip eth1 dhcp
# reboot chassis
```

Answer *yes* when you are prompted to reboot the chassis.

Steps to Set Up IxVerify Chassis Manually

The steps to setup Virtual Chassis as system level service manually are as follows:

1. Prepare an XML that can be used to define an instance of Virtual Chassis. Use the file below as a template, modify it with your own definitions for the parameters listed in Table 2.

```
$ZEBU_VTG_ROOT/scripts/runtime/ixverify/xml/ixvm-chassis.xml
```

Table 2 IxVerify Chassis Parameters

Parameter	Description
CHASSIS_NAME	This is the domain name. Example: ixvm-9.20.8379.30-chassis.
CHASSIS_QCOW2	This is the path to the qcow2 image for this Virtual Chassis instance.
PUBLIC_BRIDGE	This is the bridge for the management port. Set it to br0 unless it is created with a different name.
PRIVATE_BRIDGE	This is the virtual bridge for the test port. Set it to virbr0 unless it is created with a different name.
QEMU_GUEST_AGENT_PATH	This is the location of a UNIX socket to be created by QEMU so scripts from Virtual Ethernet can obtains information from the QEMU Guest Agent running inside the virtual machine. For example,/tmp/f16x86_64.agent. Full access to this UNIX socket is required.

2. Define and start the virtual machine. If the chassis is to be launched as a system level service, execute all `virsh` commands with sudo.

```
$ virsh define ixvm-chassis.xml
$ virsh start <domain_name>
```

Feedback

3. Login to the console of the virtual machine created:

```
$ virsh console <domain_name>
```

4. Login as an administrator (password is admin) and get to the prompt #. Execute the following at the prompt:

   a. Setup the management port with a valid static IP address, netmask and gateway:

```
# set ip eth0 <ipaddr> <netmask> <gateway>
```

   b. Setup the test port to use dhcp:

```
# set ip eth1 dhcp
```

   c. Reboot the chassis:

```
# reboot chassis
```

   d. Answer yes when you are prompted if you want to continue to reboot.

5. Wait until the Chassis is rebooted, and login as administrator again.

6. Wait for a minute for the system to be initialized. Now type *show welcome-screen* at the # prompt. You should see the *Management IPv4* in the welcome page is updated with the IP address you provide, and *Backplane IPv4* is updated with a IP address in the range of *192.168.122.2-192.168.122.254*.

```
| Management IPv4      : 10.195.1.2|
| Backplane IPv4       : 192.168.122.94|
...
```

7. Continue the setup by typing the following at the # prompt:

```
# set license-check ixverify-synopsys
# set license-server <IP address of Keysight license server>
# restart-service ixServer
```

8. After restarting ixServer, check if licenses are available:

```
# show licenses

    HostID for 10.15.214.10: 012345-619ebb-123456-7000
    HostID for localhost: 02347f-1ca437-e70050-1212

    activati | description                    | pro | qua | part |
 expi | mainte
    oncode   |                                | duc | nti | numb |
 ryda | nanced
            |                                | t   | ty  | er   | te
   | ate
            |                                |     |     |      |
   |
```

Feedback

```
 ---------+---------------------------------+-----+-----+------+---
---+-------
    16B2     | Ixia IxVerify, 1 x 800GE Port   | IxV | 512 | 939- |
 21-M | 21-May
    -C3AC-   | Subscription License BUNDLE,    | eri |     | 9584 |
 ay-2 | -2023
    4567-    | FLOATING                        | fy  |     | -01  |
 023  |
    5D55     |                                 |     |     |      |
   |
             |                                 |     |     |      |
   |
    6122-912 | IXIA IxVerify, 16-port          | IxV | 32  | 939- |
 21-M | 21-May
    3-67AB-2 | Subscription License BUNDLE,    | eri |     | 9582 |
 ay-2 | -2023
    8F7      | FLOATING - Synopsys             | fy  |     | -02  |
 023  |
             |                                 |     |     |      |
   |
    8778-    | IXIA IxVerify, 1-port           | IxV | 8   | 939- |
 21-M | 21-May
    6CAA-8F1 | Subscription License BUNDLE     | eri |     | 9586 |
 ay-2 | -2023
    7-A81D   | IxNetwork TSN protocols.        | fy  |     | -01  |
 023  |
             |                                 |     |     |      |
   |
```

9. The Virtual Chassis is fully configured. Exit *virsh* console by typing *ctrl+]*.
10. Finally, add a description to the domain created:

    ```
    $ virsh desc <domain_name> --new-desc <eth1_br>:<eth1_ip>
    ```

    For example:

    ```
    $ virsh desc ixvm-9.20.8379.30-chassis --new-desc
     virbr0:192.168.122.94
    ```

    Contact Synopsys application engineers or Keysight representatives for further assistance.

Feedback

Install Spirent TestCenter Virtual

Virtual Ethernet supports Spirent 5.33.1616 or above. Contact Spirent representative to download and install the following:

- `Spirent_TestCenter_Virtual_ChipDesign_Synopsys_QEMU_5.33.1616.qcow2`
- `Spirent TestCenter Application x64 2813.exe (MS Windows installer)`

TCP/UDP Requirements

Spirent TestCenter Client TCP/UDP Requirements

`Ingress TCP and UDP port ranges:`

- *4.97 and later: add TCP port range 9200 - 9299 to support TestCenter IQ*
- *5.05 and later: add HTTP port range 9500 - 9599 to support TestCenter IQ new Events*
- *5.05 and later: add TCP port range 9600 - 9699 to support TestCenter IQ new Events*
- *5.13 and later: add TCP port range 9700-9799 to support TestCenter IQ back-channel*
- *5.13 and later: add TCP port range 9800-9899 to support DUT Collector service for TestCenter IQ*
- *5.13 and later: add UDP/TCP port range 9900-9999 to support DUT Collector SNMP traps for TestCenter IQ*
- *5.38 and later: add UDP ports 40004 and 40005 for QUIC*

`Egress TCP and UDP port ranges:`

- *UDP port 123 NTP*
- *TCP port 445: Copy the license file from the license server towards the client. The client initiates the connection. TCP ports 27000 – 27200 (Flex-ID licensing)*
- *5.38 and later: add UDP ports 40004 and 40005 for QUIC*
- *ICMP (BLL periodically sends ICMP echo requests to the License Server.)*
- *4.97 and later: add localhost 127.0.0.1 IP address to support TestCenter IQ*

Spirent TestCenter Virtual TCP/UDP Requirements

`Ingress TCP port UDP ranges`

Feedback

- *TCP port 40004: chassis-level*
- *TCP port 51204: port group-level*
- *TCP port 1666: for STC upgrade through STC UI. (use Authentication mode to lockdown this port if required by your IT dept)*
- *TCP port 9090: for chassis controller REST API (was used by some TEMEVA apps)*
- *TCP port 22: SSH for admin console access*
- *UDP ports 40004 and 40005 for QUIC*

```
Egress TCP and UDP port ranges
```

- UDP port 123: NTP
- TCP ports 49100 – 65535: Ephemeral TCP/UDP port range
- UDP ports 40004 and 40005 FOR QUIC

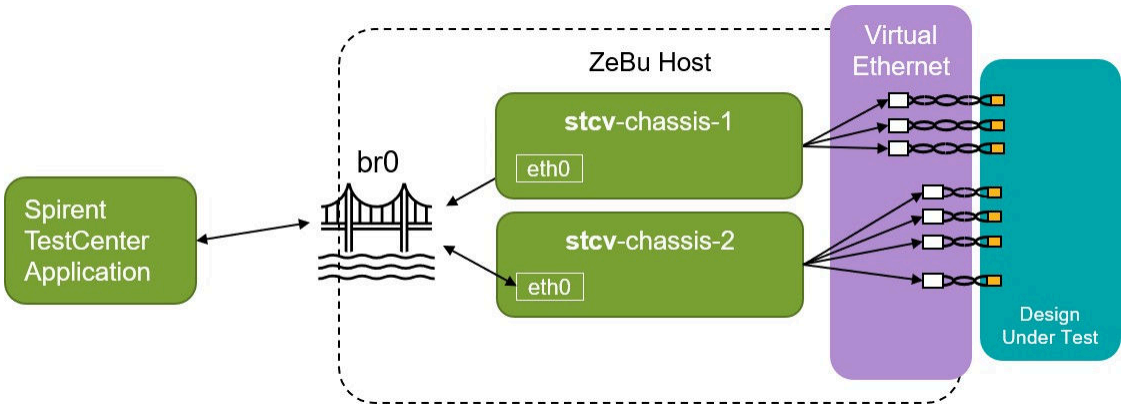
https://support.spirent.com/SC_KnowledgeView?id=FAQ10405

Spirent TestCenter Virtual Chassis Setup

Virtual Ethernet provides a framework to start the Spirent TestCenter virtual chassis in an automatic fashion. This framework involves:

- Users maintaining their own copies of qcow2 images for Virtual Chassis.
- The virtual bridge *br0* connected to physical network has already been setup on the host.
- A Spirent license server has already been setup.

Figure 11 Spirent TestCenter Virtual Chassis Setup



Feedback

For more details, see stcv in Configure YAML Fields.

Note:

Note on Port Speed

User might change the speed of each port in the Spirent TestCenter Application. When this happens, the corresponding xtor_enet_svs transactor cycles through hardware reconfiguration and might be temporarily out of service. It is important that users only apply speed change when the transactor is not transmitting or receiving, or the undefined behavior is expected.

Setting Environment Variables

The following environment variables must be properly defined for Virtual Ethernet to work:

- `ZEBU_VTG_ROOT`

Specifies the path where Virtual Ethernet Package is installed.

- `ZEBU_VTG_IXTCL`

Specifies the path where the script `ixtcl` from IxOS installation is located. IxOS from Keysight provides Tcl packages and executable that user must use to interface with virtual chassis over the network. The default install folder is as follows:

/opt/ixia/ixos-api/<version>

For example:

```
$ set ZEBU_VTG_IXTCL=/opt/ixia/ixos-api/9.20.23.6/bin/ixtcl
```

- `ZEBU_VTG_QEMU`

  Specifies the path to QEMU installation directory that is created during QEMU installation.

  For example:

  ```
  $ set ZEBU_VTG_QEMU=/opt/snps/tools/qemu-v4.1.1/
  ```

- `MANPATH`

  Add $ZEBU_VTG_ROOT/doc/man to `MANPATH` to access the manual pages:

  ```
  $ set MANPATH=$ZEBU_VTG_ROOT/doc/man:${MANPATH}
  ```

The following environment variable must be properly defined for Virtual Ethernet to support PFC:

- `XTOR_ENET_SVS_HW_PFC_EN`

4

Configure YAML Fields

A YAML configuration is a key element in Virtual Ethernet. It is prepared by users. It contains both compile time and runtime parameters critical to the operation of this solution. A YAML configuration is divided into eight sections, with each section representing a major component in Virtual Ethernet.

To start with, there are two complete examples of YAML configuration located in:

- *$ZEBU_VTG_ROOT/example/model.mii/common/keysight/example.yml*
- *$ZEBU_VTG_ROOT/example/model.mii/common/stcv/example.yml*

For details about the YAML configuration of various elements, see the following topics:

- *rpcport*
- workdir
- sync_start
- vhost_server
- ixverify
- stcv
- xtor_enet_svs
- worker

Note:

This solution supports YAML 1.2 specification, with an exception that keys with the special character ':' are not allowed. For example, the following YAML document is legal as per the 1.2 specification but not with Virtual Ethernet:

```
Monday:
          8:00: "Breakfast with kids"
          9:30: "Conference call with Joe"
```

The syntax of YAML specification can be found at https://yaml.org/ .

Feedback

rpcport

This specifies the port number that Virtual Ethernet RPC server listens to. For example:

```
rpcport: 31517
```

If not specified, or it is set to 0, Virtual Ethernet finds a free TCP port number to launch the Virtual Ethernet RPC server. Otherwise, Virtual Ethernet attempts to start a RPC server with the given port number. Note that Virtual Ethernet reports an error and exits if it fails to start a RPC server, either because the given port number is already used, or there is no free TCP port found.

workdir

This specifies the working directory for Virtual Ethernet. For example:

```
workdir: "./vtg.work"
```

Virtual Ethernet reports an error and exits if a work directory cannot be removed and then created.

sync_start

This specifies a staggered delay time in emulation time. The default time is 250 microseconds. In the following example, the staggered start delay is specified as 25 microseconds:

```
sync_start:
  staggered_delay: 25e-6
```

Note:
staggered start is not relevant when Spirent TestCenter Virtual is deployed.

Limitation of the Staggered Start Feature

For staggered start, there is a *Margin of Error (MOE)* specified in number of clocks.

```
moe = 1562.5 / GBX clock freq
```

where `moe` denotes the number of clock cycles that the sof can vary from expected values. See the following help page for more details: `<ZEBU_IP_ROOT>/xtor_enet_svs/doc/html/Clock_page.html`

In addition, the staggered start feature is not supported for 1G and lower speeds. Consider the following examples.

Feedback

For 50G, `GBX clock freq = clk_195_3125_i / clk_390_625_i / clk_781_25_i moe = 8 / 4 / 2`

For 25G, `GBX clock freq = clk_97_65625_i/clk_195_3125_i/... moe = 16 / 8 / ..`

vhost_server

This specifies the settings for a virtual host server required for IxVerify or Spirent TestCenter Virtual. The settings include socket memory and the directory for hugepages. Here is an example of how to specify the vhost_server section:

```
vhost_server:
  socket_mem: "1024,1024"
  hugepage_dir: "/dev/shm"
```

Above is a YAML data structure with a list of elements under the key "vhost_server". Each element is a mapping to a pair of key and value:

socket_mem

This specifies the amount of memory allocated for Virtual Ethernet on a per-socket basis. Here, the socket refers to a physical processor socket on the underlying hardware. Allocating memory on a per-socket basis can improve memory access performance. The socket_mem parameter is specified in megabytes (MB) and can be set either globally or for each individual NUMA (Non-Uniform Memory Access) node.

hugepage_dir

This specifies the mount point that Virtual Ethernet can use to allocate and use memory pages by calling appropriate system calls.

If the path specified is not of *hugetlbfs* type and therefore not a hugepage mount point, *Virtual Ethernet* falls back to use standard memory (also known as small pages) available in the system. Standard memory refers to the memory that is allocated by the operating system in small chunks of 4KB.

Note:

Using standard memory instead of hugepages might negatively impact the performance of this solution.

It should be noted that the default setting, */dev/shm*, is not a hugepage mount point and is instead a special file system that enables efficient memory sharing between programs. This is the default setting if the user has not yet set up Hugepage Setup on the system.

Feedback

However, using standard memory instead of hugepages may have a detrimental impact on the performance of this solution.

ixverify

The `ixverify` section in the YAML data structure specifies the configuration of IxVerify, including the ID, Ethernet addresses, virtual bridge, and the number of virtual load modules (VLMs). For example:

```
ixverify:
  chas:
    - id: 1
      #domain: "ixvm-9.20.8739.30-chassis"
      eth: ["10.194.163.84", "192.168.122.111"]
      virbr: "virbr0"
      card:
      - {id: 11, nb_ports: 6, img: /u/vtg/9.20.8739.30/vlm-11.qcow2}
      - {id: 12, nb_ports: 6, img: /u/vtg/9.20.8739.30/vlm-12.qcow2, pfc:
 true}
```

The `ixverify` section is a list of elements under the key ixverify. Each element is a mapping with the following keys and values:

- chas
- id
- eth, virbr
- domain
- card

chas

It points to a YAML sequence with each member in the sequence specifying a unique IxVerify Chassis.

id

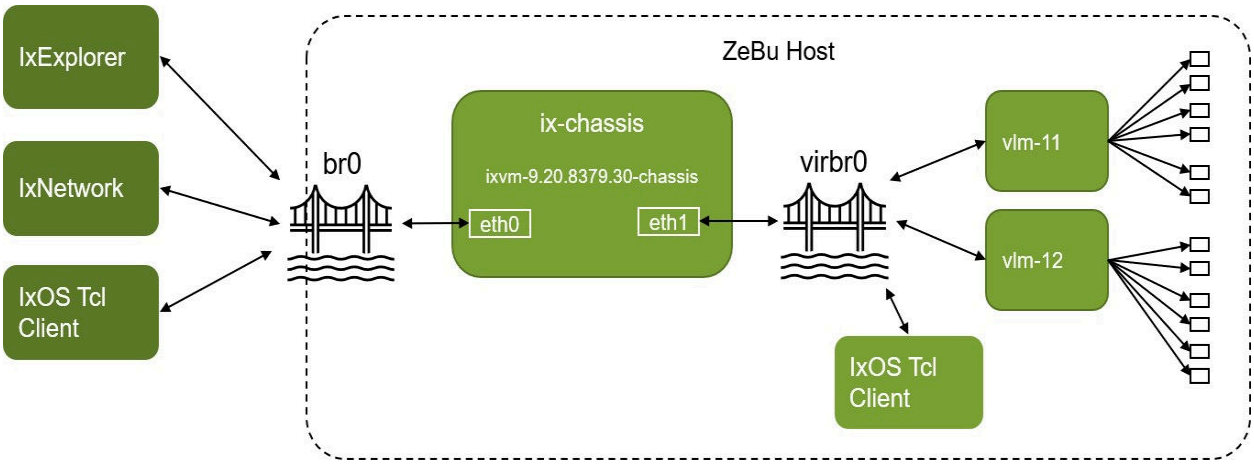
An integer representing the Chassis identifier. The chassis ID, together with card ID and port ID, is used during port mapping of a virtual port from IxVerify to a xtor_enet_svs transactor in the ZeBu model.

Feedback

eth, virbr

Each Virtual Chassis has two network interfaces: eth0 (the management port) and eth1 (the test port). It is typical for the Chassis to be connected to a user interface (for example, IxExplorer, IxNetwork) over its management port via a network bridge br0, and its virtual load module(s) over its test port via the virtual bridge virbr0.

Figure 12 Virtual Chassis network interfaces



Note:

Users are responsible for creating the virtual bridges, preparing static IP addresses for the management port of the Chassis, and setting up the Chassis in this flow.

When the Chassis is setup manually (that is, user login to the Chassis and setup the network device for both management and test port, license setup plus other configurations), user should specify the parameters eth and virbr in this YAML configuration so Virtual Ethernet knows how to connect the virtual load modules, which are launched by Virtual Ethernet, to the Virtual Chassis.

Moving forward, you may use scripts provided by Virtual Ethernet to set up and launch the Chassis in a semi-automatic fashion. When this happens, eth and virbr are no longer needed, and they are replaced by domain.

domain

A string representing the domain name of the Chassis. It is an attempt to eliminate the need for setting up *eth* and *virbr* in a YAML file by using scripts from Virtual Ethernet to setup and launch the Virtual Chassis. The script achieves automation by asking user for Chassis specific parameters upfront, setting up and launching the Chassis as per user

inputs, discovering the IP address of the test port at runtime, and adding a description to this virtual machine using the following command:

```
$ virsh desc <eth1_br>:<eth1_ip>
```

Where,

- *eth1_br* is the bridge connected to the test port, same as virbr.
- *eth1_ip* is the IP address of the test port, which is discovered automatically.

These information is similar to what you specify with the parameters*eth* and *virbr* previously. The only difference is that the user no longer needs to specify them in the YAML file.

The Chassis setup script is located at *$ZEBU_VTG_ROOT/example/tests/tcl/client.tcl*. Follow the description in a Virtual Chassis Launcher Script for details.

---

card

It points to a YAML sequence, with each member in the sequence specifying an unique IxVerify VLM.

id

An integer representing the Card identifier. The card ID, together with the Chassis ID and a port ID, is used during port mapping of a virtual port from IxVerify to a*xtor_enet_svs* transactor in the ZeBu model.

nb_ports

Number of ports in this card. The maximum number of ports allowed is 25. The more ports are created, the more CPU resources are required. If user specifies 2 ports, the first port has a port ID of 1 and the second port has a port ID of 2.

img

The points to the Virtual Load Module qcow2 image for the this particular card. Each card must have its own qcow2 image.

Contact Keysight representative for a copy of the qcow2 image. If user wants to run with two virtual load modules, just copy the golden image and use it for the second virtual machine.

Feedback

pfc

PFC support is available with Virtual Ethernet Keysight only. To enable PFC for Virtual Ethernet, perform the following steps:

1. Compile the `xtor_enet_svs` transactor with the VCS switch `(+define +XTOR_ENET_SVS_HW_PFC_EN)`.
2. In the YAML file passed to `vtg::init` (in Tcl) or `SNPS::VSA::VTG::init()` (in C++), in the `ixverify` section, under `card` specifications, add the `pfc` field and set it to `true`. To see the example usage, scroll to reach the top of the `ixverify` section.

stcv

Specifies how Spirent TestCenter Virtual (STCv) should be setup, including its ID, IP address and virtual bridge. In Spirent TestCenter Virtual, Chassis is a virtual machine which is responsible for traffic generation and analysis. One chassis can has up to 16 ports. User may setup multiple chassis and ports depending on their needs and preferences.

Here is an example on how the stcv section is specified:

```
stcv:
  chas:
  - id: 1
    eth: "10.194.163.15"
    virbr: "br0"
    img:
 "/u/vtg/Spirent_TestCenter_Virtual_ChipDesign_Synopsys_QEMU_5.33.1616.c1
.qcow2"
    nb_ports: 6
  - id: 2
    eth: "10.194.163.16"
    virbr: "br0"
    img:
 "/u/vtg/Spirent_TestCenter_Virtual_ChipDesign_Synopsys_QEMU_5.33.1616.c2
.qcow2"
    nb_ports: 4
```

Above is a YAML data structure defining a list of elements under the key "stcv". Each element is a mapping with the following keys and values:

chas

It points to a YAML sequence, with each member in the sequence specifying an unique STCv Chassis.

Feedback

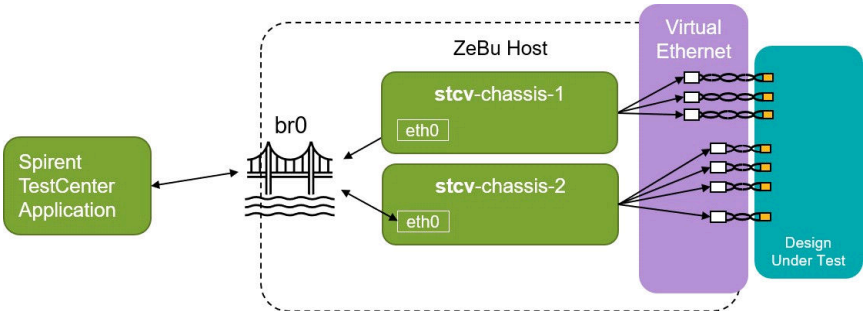
id

An integer representing the Chassis identifier. The card ID together with a port ID is used during port mapping from a STCv virtual port to a *xtor_enet_svs* transactor in the ZeBu model.

eth, virbr

It is typical for the Chassis to be connected to a user interface (for example, Spirent TestCenter Application) over its management port.

Figure 13 Spirent TestCenter Application



User uses the parameter *eth* to specify the IP address assigned to the Chassis, and *virbr* to specify over which network bridge a Chassis is connected to its client applications.

Note that user is responsible for creating the virtual bridges, preparing static IP addresses and setting them up for the Chassis.

nb_ports

Number of ports in this chassis. The maximum is 16. The more ports are created, the more CPU resources are required. If user specifies 2 ports, the first port has a port ID of 1 and the second port has a port ID of 2.

img

This points to the qcow2 image for the given chassis. Each chassis must have its own qcow2 image.

Contact Spirent representative for a copy of the qcow2 image. If user wants to run with two chassis, just copy the golden image and use it for the second chassis.

Feedback

xtor_enet_svs

Specifies details of transactors xtor_enet_svs in the ZeBu model, including their HDL paths, initial hardware configurations, and worker ID. Here is an example of a ZeBu model with 6 xtor_enet_svs transactors instantiated:

```
xtor_enet_svs:
- {hdlpath: "hw_top.enet[0].inst", chas: 1, card: 11, port: 1, hwconfig:
 *hw1, worker: 0}
- {hdlpath: "hw_top.enet[1].inst", chas: 1, card: 11, port: 2, hwconfig:
 *hw1, worker: 0}
- {hdlpath: "hw_top.enet[2].inst", chas: 1, card: 11, port: 3, hwconfig:
 *hw1, worker: 0}
- {hdlpath: "hw_top.enet[3].inst", chas: 1, card: 11, port: 4, hwconfig:
 *hw1, worker: 1}
- {hdlpath: "hw_top.enet[4].inst", chas: 1, card: 11, port: 5, hwconfig:
 *hw1, worker: 1}
- {hdlpath: "hw_top.enet[5].inst", chas: 1, card: 11, port: 6, hwconfig:
 *hw1, worker: 1}
```

Above is a YAML data structure defining a list of six elements under the key*xtor_enet_svs*. Each element is a mapping for a particular transactor instance with the following keys and values:

hdlpath

A string representing a HDL path of the transactor instance in the ZeBu model.

chas, card, port

A set of integers representing the virtual port this transactor is bound to.

In the example, the transactor with hdlpath *hw_top.enet[0].inst* is to be connected to a virtual port identified as chassis 1, card 11, and port 1.

Note:

For Spirent TestCenter virtual, do not specify 'card' as each virtual port is identified as a {chassis, port} pair.

hwconfig

A map that defines the initial configurations to be applied to the given transactor. In the YAML data structure above, a reference to a YAML anchor named "hw1" (defined

elsewhere in the YAML document) is used. Users can also specify initial configurations without using a YAML anchor. For example:

```
xtor_enet_svs:
- {hdlpath: "hw_top.enet[0].inst", hwconfig: {ENET_SPEED: 400.0}, worker:
 0}
```

Note: See*$ZEBU_IP_ROOT/doc/html/Configure_page.html* for all possible configurations supported by *xtor_enet_svs*.

---

worker

An integer representing the worker number.

Workers are pthread spawned by Virtual Ethernet to serve transactors. Transactor has no worker number specified means it is inactive. Multiple transactors can share a worker. It is recommended for transactors configured to run at low rate (for example, *ENET_SPEED <= 200*). On the other hand, transactors configured to run at high rate (for example, *ENET_SPEED >= 800.0*) should have a dedicated worker. The exact transactor-worker setup depends on the performance requirements and number of CPU resources available on the host.

---

clkfreq

The frequency, specified as MHz, of the clock signal connected to *clk_1562_5_i* input of the*xtor_enet_svs* transactor. Default is 1562.5.

A Note on Transactor ID (XID)

Transactors are assigned with a XID based on the order at which they are specified in the*xtor_enet_svs* section of YAML configuration. In the following example, the transactor with the hdlpath of "bbb" has an XID of 1.

```
xtor_enet_svs:
- {hdlpath: "aaa", hwconfig: *hw1, worker: 0}
- {hdlpath: "bbb", hwconfig: *hw1, worker: 1}
```

However, if the order are reversed like this:

```
xtor_enet_svs:
- {hdlpath: "bbb", hwconfig: *hw1, worker: 1}
- {hdlpath: "aaa", hwconfig: *hw1, worker: 0}
```

The transactor with a hdlpath of "*bbb*" has an XID of 0 instead.

Feedback

Virtual Ethernet has a big collection of Tcl commands for transactors *xtor_enet_svs* specified in the YAML configuration. The first argument to these Tcl commands are XID, which stands for transactor ID. Here is an example:

```
vtg>> vtg::xtor::has_pcs 0
```

The command above checks if the transactor with XID=0 has a PCS interface or not.

sem_mode

A new field must be added to enable the Simulated Emulation or Simulation of Emulation (SEM) mode.

```
sem_mode:
  xtor_dpi_config: "./zcui.work/zebu.work"

#xtor_dpi_config: Share the path of directory where xtor_dpi.lst is
 present.
```

Note:
SEM is supported in zRci mode only.

For reference example, use the following path:

```
$ZEBU_VTG_ROOT/example/model.mii/common/sem/keysight/example.yml
```

worker

The "worker" section in the YAML configuration allows users to pin a worker to a specific CPU set in the host machine. This can be useful for optimizing performance and resource utilization in multi-core systems.

If the "worker" section is not provided, the workers are allowed to run on any available CPU in the host machine. On the other hand, if the user specifies the "worker" section, then each worker spawned by Virtual Ethernet is pinned to the specified CPU set.

To specify the CPU set for a worker, the user should include a "worker" section in the YAML configuration file. Each "worker" should contain a list of CPU cores or CPU sets where the worker can run. For example:

```
worker:
0: {cpuset: 1-4}
```

In this example, the worker with WID=0 can run on any of the first four CPU cores on the host machine.

It is important to note that pinning workers to specific CPU cores can improve performance in some cases, but it can also limit flexibility and resource sharing in multi-core systems. Therefore, users should carefully consider the trade-offs before pinning workers to specific CPU sets.

5

User-Defined Enqueue and Dequeue Callbacks

You can define your own dequeue and enqueue callback functions. The dequeue function generates Ethernet frames and metadata in memory space provided by Virtual Ethernet. Whereas the enqueue function lets Virtual Ethernet receive Ethernet frames from `xtor_enet_svs` and store the frames along with metadata in local memory space.

See the following sections for more information.

Enqueue

Enqueue is the mechanism that lets you access Ethernet frames received by a `xtor_enet_svs` transactor. The process starts with Virtual Ethernet allocating frame buffers. When the transactor receives an Ethernet frame, it notifies Virtual Ethernet, which stores the frame in its preallocated frame buffers.

Virtual Ethernet calls the user-provided enqueue function when any one of the following conditions is met:

- When Virtual Ethernet has buffered 32 Ethernet frames provided by `xtor_enet_svs`, or
- When an alarm goes off, which is set to ring every 250 miniseconds.

Note:
See the *ZeBu Virtual Ethernet Tcl Commands Reference Guide* for details on how to register and unregister dequeue and enqueue callback functions.

Function Prototype

A user-provided enqueue function must have the following function prototype:

```
int (*cb) (const char* buf_array[], const int buf_avail, const void*
 context);
```

Feedback

Arguments

A user provided enqueue function expects the following arguments:

- `buf_array`: It is an array of pointers, with each entry pointing to a frame buffer, which starts with a metadata data structure followed by frame payload. The metadata data structure contains information about the Ethernet frames received by the transactor.
- `buf_avail`: It indicates how many frame buffers are available for user to consume. Note that the enqueue function must consume all the buffers provided.
- `context`: It is a pointer to the user-defined context when an enqueue function is registered.

Return

The enqueue function should return the number of buffers it has consumed. Since the enqueue function must consume all the buffer provided, the return value should always be the same as `buf_avail`.

Dequeue

Dequeue is the mechanism that lets you provide Ethernet frames for the `xtor_enet_svs` transactor to transmit.

The process starts with Virtual Ethernet allocating frame buffers, followed by Virtual Ethernet calling the user-provided dequeue functions periodically. When the dequeue function has frames to send, it populates the buffers provided by Virtual Ethernet. The function notifies Virtual Ethernet with the number of buffers that are populated as a return value. After the dequeue function returns with some Ethernet frames, Virtual Ethernet passes them to the transactor for transmission.

Function Prototype

A user-provided dequeue function must have the following function prototype:

```
int (*cb) (const char* buf_array[], const int buf_avail, const void*
 context);
```

Arguments

A user-provided dequeue function expects the following arguments:

- `buf_array`: It is an array of pointers, with each entry pointing to a frame buffer. User is responsible for populating that memory block with a metadata data structure, followed by frame payload for transmission.
- `buf_avail`: It indicates how many frame buffers are available for user to populate. Note that not all buffers have to be populated. The number of buffers populated should be returned to the caller.
- `context`: It is the pointer to a user-defined context when the dequeue function is registered.

Return

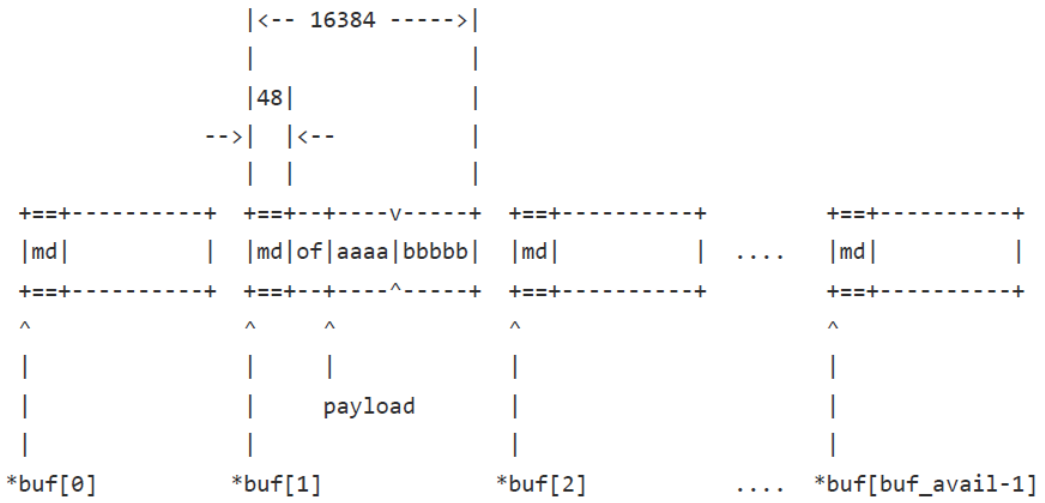
The dequeue function should return the number of frame buffers it has populated, and it should be equal or less than `buf_avail`. If user has no frames to send, just return 0.

Memory Layout

Each frame buffer occupies 16384 bytes. The first 48 bytes are identified as metadata and the rest are frame data.

- md = metadata, 48 bytes
- of = offset to payload (0, normally)
- aa.. = frame payload, max of (16384 -48) bytes
- bb.. = dead space before the start of the next buffer

Feedback



Metadata

The first 48 bytes in a buffer is a metadata, which specifies attributes that may not be payload related.

```
typedef struct mdata_s {
       uint16_t payload_size;
       uint8_t generic_flags; // 0: data packet; 1: control packet
       union {
              uint64_t dequeue_flags; // populated by user-provided
 dequeue function.
              uint64_t enqueue_flags; // to be consumed by user-provided
 enqueue function.
       };
       union {
         uint64_t inter_frame_gap; // populated by user-provided dequeue
 function.
         uint64_t emulator_time; // to be consumed by user-provided
 enqueue function.
       };
       uint8_t offset; // The offset from the end of metadata to payload.
                       // calculated automatically by vTG during enqueue.
                       // in case Huge memory is in use, otherwise,
                       // default is 0.
       char rsvd[21];
       } __attribute__((packed)) mdata_t;
```

Feedback

payload size

Payload size, specified in bytes. By default, the transactor inserts a 4-byte FCS and therefore, 'payload_size' should not include the 4-byte FCS. There are cases that users want the transactor to send an Ethernet frame with a user provided FCS. In this case, user should set bit 9 of "dequeue_flags" to 1, include the 4-byte FCS as part of the payload, and 'payload_size' should include the FCS.

generic_flags

The field can be set by both user provided enqueue / dequeue function and Virtual Ethernet.

- 0: This is a data buffer. It contains payload.
- 1 : This is a control-only buffer. It contains no payload.

dequeue_flags

The dequeue_flags field is only available to dequeue function. It is populated by the user-provided dequeue function and consumed by Virtual Ethernet.

Bit	*Mask*	*Description*
0	0x1	Only used for data buffer. If set, the field "inter_frame_gap" contain valid value. *
3	0x8	Only used for control buffer. If set, the transactor enters free running mode. **
7	0x80	Only used for data buffers. If set, the transactor does not insert a timestamp. *
8	0x100	Only used for data buffers. If set, transactor inserts a bad CRC.
9	0x200	Only used for data buffers. If set, transactor does not insert a CRC.

Note:

* User must set bit 0 and bit 7 set in the current version.

** User must generate a control-only packet (that is, generic_flags=1) with bit 3 set at the end of transmission.

enqueue_flags

The field is only available to enqueue function; it is populated by Virtual Ethernet and consumed by user-provided enqueue function.

Feedback

Bit	*Mask*	*Description*
2	0x4	Used for both data and control buffers. If set, the field "emulator_time" contains valid value.
8	0x 100	Only used for data buffers. If set, this frame is corrupted.

inter_frame_gap

The field is only available to dequeue function. The user-provided dequeue function populates this field and Virtual Ethernet consumes this field. Only used for data buffers, with bit 0 of `dequeue_flags` set. It specifies the number of idles inserted before sending the payload. The minimum is 12.

emulator_time

The field is only available to enqueue function. Virtual Ethernet populates this field and the user-provided enqueue function consumes the field. This field is used in both control and data buffers with bit 2 of `enqueue_flags` set.

- For control-only buffer, it specifies the current emulator time (in ps).
- For data buffer, it specifies the emulator time (in ps) at which the Start of Packet (SOP) is detected.

6

RPC Server and User API

The Virtual Ethernet software automatically launches a remote protocol call (RPC) server when it is started, which listens to the TCP port specified in the Virtual Ethernet YAML configuration. This server enables users to issue commands from a remote client via the RPC interface.

Support is available for both TCL and Python clients, although the Python client offers a limited set of APIs.

The following sections list Tcl and Python commands that are currently supported:

- General Commands Supported by ZeBu
- Virtual Ethernet System-Level Commands
- Commands to Access Virtual Ethernet System
- Transactor Specific Low-Level Commands (Implemented by xtor_enet_svs)
- Transactor Specific High-Level Commands (Implemented by Virtual Ethernet)
- Logging-Related Commands
- Miscellaneous Commands

For more information on these Tcl commands, refer to the *ZeBu Virtual Ethernet Tcl Commands Reference Guide*.

For more information on Python APIs, refer to Python APIs.

General Commands Supported by ZeBu

Table 3 General Tcl Commands Supported by ZeBu

Command	Description
`zebu::runmgr`	Returns zebu time information via ZEBU::RunManager
`zebu::signal`	Accesses a HDL signal in a ZeBu model
`zebu::fwc`	Gives a fast waveform capture

Feedback

Table 3 General Tcl Commands Supported by ZeBu (Continued)

`zebu::wavefile`	Captures waveforms for dynamic probes
`zebu::tcleval`	Evaluates a Tcl command on the ZeBu host
`zebu::version`	Gets the version of ZeBu components
`zebu::readfile`	Reads a text file
`zebu::bash`	Executes bash commands on the remote ZeBu host
`zebu::sniffer`	Captures emulator states and records stimuli in multiple snapshots or frames automatically

Virtual Ethernet System-Level Commands

Table 4 Virtual Ethernet System-Level Tcl Commands

Command	Description
`vtg::system::start`	Starts worker threads and virtual machines
`vtg::system::stop`	Stops worker threads and virtual machines
`vtg::system::ixsyncserver`	Starts or stops IxVerify sync server
`vtg::system::virtmgr`	Executes a virtmgr command
`vtg::system::xtormgr`	Executes a xtormgr command
`vtg::system::workermgr`	Executes a worker manager command
`vtg::system::worker`	Executes a worker command

Table 5 Virtual Ethernet System-Level Python Commands

Command	Description
`vtg.system.start()`	Starts worker threads and virtual machines
`vtg.system.stop()`	Stops worker threads and virtual machines
`vtg.system.workermgr()`	Executes a worker manager commands

Commands to Access Virtual Ethernet System

Table 6 Commands to Access Virtual Ethernet System Configuration

Command	Description
`vtg::ymlcfg::getopt`	Gets the YAML presentation of a given YAML node
`vtg::ymlcfg::getsize`	Gets the number of elements of a given YAML node
`vtg::ymlcfg::addopt`	Turns a YAML node into a sequence by adding a new node to the end
`vtg::ymlcfg::setopt`	Modifies an existing or adds a new YAML node
`vtg::ymlcfg::delopt`	Erases a YAML node
`vtg::ymlcfg::dump`	Dumps a YAML document
`vtg::ymlcfg::reset`	Resets a node to its initial setting

Transactor Specific Low-Level Commands (Implemented by xtor_enet_svs)

Table 7 Transactor Specific Low-Level Tcl Commands (implemented by xtor_enet_svs)

Command	Description
`vtg::xtor::ll::set_debug_level`	Calls xtor_enet_svs's setDebugLevel() API
`vtg::xtor::ll::print_dashboard`	Calls xtor_enet_svs's printDashboard() API
`vtg::xtor::ll::set_system_time`	Calls xtor_enet_svs's setSystemTime() API
`vtg::xtor::ll::register_cb_pcs`	Calls xtor_enet_svs's registerPcsStatusCb() API
`vtg::xtor::ll::insert_rs_fec_error`	Calls xtor_enet_svs's insertRSFECError() API
`vtg::xtor::ll::insert_rs_transcode_error`	Calls xtor_enet_svs's insertRSTranscodeError() API
`vtg::xtor::ll::insert_am_error`	Calls xtor_enet_svs insertRSAMError() API
`vtg::xtor::ll::load_user_am`	Calls xtor_enet_svs's loadUserAM() API

Table 7 Transactor Specific Low-Level Tcl Commands (implemented by xtor_enet_svs) (Continued)

`vtg::xtor::ll::print_cfg_params`	Calls xtor_enet_svs's printCfgParams() API
`vtg::xtor::ll::set_config`	Calls xtor_enet_svs's setConfig() API
`vtg::xtor::ll::get_config`	Calls xtor_enet_svs's getConfig() API
`vtg::xtor::ll::get_cycle_count`	Call xtor_enet_svs's getCycleCount() API
`vtg::xtor::ll::get_frame_count`	Call xtor_enet_svs's getFrameCount() API
`vtg::xtor::ll::get_config_type`	Call xtor_enet_svs's getConfigParamType() API
`vtg::xtor::ll::get_config_list`	Call xtor_enet_svs's getEnetConfigParamList() API
`vtg::xtor::ll::get_config_default`	Call xtor_enet_svs's getConfigParam() API
`vtg::xtor::ll::access_backdoor_mdio_rd XID [OPTION …]`	Calls xtor_enet_svs's accessBackdoorMdioRd() API
`vtg::xtor::ll::access_backdoor_mdio_wr XID [OPTION ...]`	Calls xtor_enet_svs's accessBackdoorMdioWr() API
`vtg::xtor::ll::send_mdio_txn XID [OPTION ...]`	Calls xtor_enet_svs's sendMDIOTxn() API

Table 8 Transactor Specific Low-Level Python Commands (implemented by xtor_enet_svs)

Command	Description
`vtg.xtor.ll.print_dashboard()`	Calls `printDashboard()` API of the Ethernet transactor
`vtg.xtor.ll.set_config ()`	Calls `setConfig()` API of the Ethernet transactor
`vtg.xtor.ll.get_config ()`	Calls `getConfig()` API of the Ethernet transactor

Feedback

Transactor Specific High-Level Commands (Implemented by Virtual Ethernet)

Table 9 Transactor Specific High-Level Tcl commands (implemented by Virtual Ethernet)

Command	Description
`vtg::xtor::has_pcs`	Reports if a transactor has a PCS
`vtg::xtor::is_800G`	Reports if a transactor is a 800G or a 400G driver
`vtg::xtor::pcs_monitor`	Accesses PCS status monitor
`vtg::xtor::get_pcsstatus`	Gets PCS status from xtor_enet_svs's hardware
`vtg::xtor::get_dashboard`	Gets xtor_enet_svs's dashboard with optional update from hardware
`vtg::xtor::get_pid0`	Gets xtor's pid0 in the form of {chas_id card_id port_id}
`vtg::xtor::clr_dashboard`	Clears transactor statistics
`vtg::xtor::speedtest`	Built-in speedtest for regular traffic
`vtg::xtor::has_pfc`	Checks if the transactor is compiled with Priority Flow Control (PFC)-enabled hardware
`vtg::xtor::pcaprp`	Accesses PCAP replay scheduler
`vtg::xtor::set_hwconfig`	Reloads hardware configuration in the system configuration for the transactor
`vtg::xtor::wait_pcs_lock`	Waits for the given list of transactors to reach PCS lock
`vtg::xtor::speedtest_ptp`	Built-in speedtest for PTP
`vtg::xtor::speedtest_fpe`	Built-in speedtest for Frame Preemption
`vtg::xtor::ts1588_queue`	Accesses the 1588 timestamp (t1,t4) queue
`vtg::xtor::fps_monitor`	Accesses the FPS (frame-per-second) reporter
`vtg::xtor::hex_dumper`	hex dump Ethernet frames to screen or file
`vtg::xtor::pcap_dumper`	Captures Ethernet frames into PCAP file
`vtg::xtor::rx_capture`	Accesses RX capture buffer

Feedback

Table 9 Transactor Specific High-Level Tcl commands (implemented by Virtual Ethernet) (Continued)

`vtg::xtor::set_hlparam`	Sets high-level parameters
`vtg::xtor::get_hlparam`	Gets high-level parameter
`vtg::xtor::reset_hlparam`	Resets high-level parameters
`vtg::xtor::register_cb_eq`	Registers user-defined enqueue callback function
`vtg::xtor::unregister_cb_eq`	Removes registered user-defined enqueue callback function
`vtg::xtor::register_cb_dq`	Registers user-defined dequeue callback function
`vtg::xtor::unregister_cb_dq`	Removes registered user-defined dequeue callback function
`vtg::xtor::register_cb_rx`	Registers a user-defined RX callback function
`vtg::xtor::unregister_cb_rx`	Removes a previously registered user-defined RX callback function
`vtg::xtor::register_cb_tx`	Registers a user-defined TX callback function
`vtg::xtor::unregister_cb_tx`	Removes a previously registered user-defined TX callback function

Table 10 Transactor Specific High-Level Python Commands (implemented by Virtual Ethernet)

Command	Description
`vtg.xtor.get_pcsstatus()`	Gets PCS status from the Ethernet transactor hardware

Logging-Related Commands

Table 11 Logging-Related Tcl Commands

Command	Description
`vtg::logger::list`	Returns the list of loggers available
`vtg::logger::new`	Creates a new logger

Feedback

Table 11 Logging-Related Tcl Commands (Continued)

`vtg::logger::delete`	Deletes an existing logger
`vtg::logger::msg`	Logs debug messages
`vtg::logger::set_verbosity`	Sets the verbosity of a given logger
`vtg::logger::get_verbosity`	Gets the verbosity of a given logger
`vtg::logger::file_create`	Opens a text file for writing
`vtg::logger::file_append`	Opens a text file for appending
`vtg::logger::file_close`	Closes a previously opened text file
`vtg::logger::file_list`	Lists all files currently opened by a logger
`vtg::logger::set_scope`	Specifies the scope of a logger

Table 12 Logging-Related Python Commands

Command	Description
`vtg.logger.new()`	Creates a new logger
`vtg.logger.list()`	Returns the list of loggers available
`vtg.logger.msg()`	Logs debug messages

Miscellaneous Commands

Table 13 System Initialization and Client Connection Tcl Commands

Command	Description
`vtg::connect`	Connects to Virtual Ethernet RPC server hostname and port number
`vtg::init`	Initializes Virtual Ethernet
`vtg::help`	Displays help messages

Feedback

Table 14 System Initialization and Client Connection Python Commands

Command	Description
`vtg.connect()`	Gets PCS status from Ethernet transactor hardware
`vtg.vtgHelp()`	Displays help messages

7

Virtual Ethernet Examples

This chapter describes how to compile and run the example that comes with the Virtual Ethernet package. The source code, scripts, and required files are located in the directory ${ZEBU_VTG_ROOT}/example.

Refer to the following structure of the example directory:

```
├── ex1-zebu
│   ├── Makefile
│   └── testbench.cc
├── ex2-zrci
│   ├── debug
│   ├── Makefile
│   ├── run.tcl
│   └── testbench.cc
├── ex-sem
├── ngtb-example
├── model.mii
│   ├── common
│   ├── sem
│   └── zebu
├── model.pcs
│   ├── common
│   ├── sem
│   └── zebu
├── tests
│   ├── etc
│   └── tcl
└── utils
    ├── pcs_monitor.bin
    └── register_cb_pcs
```

Ethernet Transactor

The Ethernet (*xtor_enet_svs*) transactor is responsible for delivering Ethernet packets generated by third-party traffic generators to the DUT. The *xtor_enet_svs* transactor has four different flavors, each with its own Verilog module definition, supporting different functionalities such as interface type and supported rate.

Feedback

The four flavors of Ethernet transactors and their corresponding descriptions are as follows:

Flavor	Description
`xtor_enet_800G_svs`	Supports MII with speeds ranging from 10Gbps to 1.6Tbps.
`xtor_enet_400G_svs`	Supports MII with speeds ranging from 10Mbps to 400Gbps.
`xtor_enet_pcs_800G_svs`	Supports PCS with speeds ranging from 10Gps to 1600Gbps.
`xtor_enet_pcs_400G_svs`	Supports PCS with speeds ranging from 1Gbps to 400Gbps.

Note:
To enable 1.6Tbps, you need to set the parameter `max_speed` to `1600`.

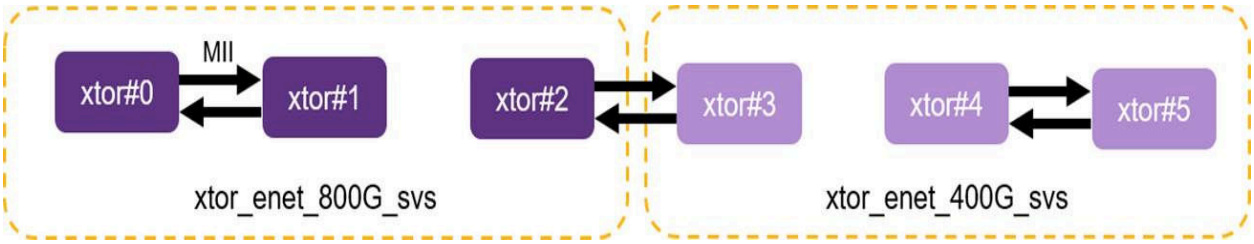
```
// set maximum speed to 1.6Tbps
defparam my_xtor_instance.max_speed = 1600;
```

This example includes two HDL testbenches. The first testbench, located in *$ZEBU_VTG_ROOT/example/model.mii*, instantiates the MII flavor transactors. The second testbench, located in *$ZEBU_VTG_ROOT/example/model.pcs,* instantiates the PCS flavor transactors.

model.mii

Within the Verilog testbench located at *$ZEBU_VTG_ROOT/example/model.mii/common/rtl_v/hw_top.sv*, there are six *xtor_enet_svs* transactors instantiated. Of these transactors, the first three are *xtor_enet_800G_svs*, while the remaining three are *xtor_enet_400G_svs*. These instances are connected in a back-to-back fashion, where adjacent ports are connected to each other.

Figure 14 model.mii

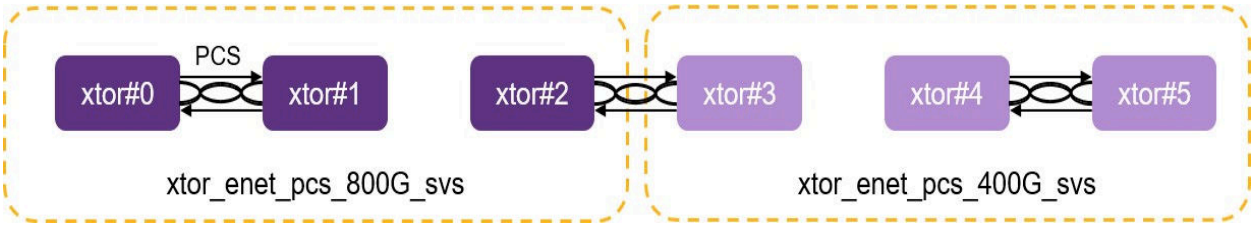


Feedback

model.pcs

A similar topology can be found in the Verilog testbench located at *$ZEBU_VTG_ROOT/example/model.pcs/common/rtl_v/hw_top.sv* . The only difference is that the MII flavor transactors are replaced with their PCS counterparts.

Figure 15 model.pcs



Limitation in Latency for PCS Drivers

All latency reported is currently relative to the *xMII* interface. For PCS users, the latency is reported as an aggregate of the latencies of TX PCS, RX PCS, and that of the chip.

ZeBu Example

Building a ZeBu Model

During model build, HDL code is first synthesized into a gate-level netlist, which describes the logical connections between different hardware components in the design. The netlist is then mapped to the corresponding components in the ZeBu technology library. After the initial mapping is complete, the netlist is further optimized for area, speed, and power consumption. Finally, the optimized netlist is loaded onto the ZeBu emulator, which simulates the behavior of the hardware design at a speed much faster than CPU-based simulator.

To build a ZEBU model, perform the following steps:

For MII:

```
$ cd example/model.mii/zebu; make
```

For PCS:

```
cd example/model.pcs/zebu; make
```

The 'make' process continues to finish compiling DPI transactor shared objects required for ZeBu emulation.

User may also compile the shared objects for DPI transactors manually:

```
$ make -C zcui.work/backend_default -f dpixtor.make
```

Building a Software Testbench

A ZeBu model is basically a netlist that can be loaded to the ZeBu emulator. User also needs building a ZeBu/transactor-aware software, which lets user to interact with the model running in the emulator.

Here are a few examples which show what may be included in the software testbench:

- hook to load data a text file to a HDL memory
- hook to create waveform for part of the emulated hardware
- hook to xtor_enet_svs so user can instruct it to send user-defined Ethernet packets
- hook to Virtual Ethernet over its API (see *$ZEBU_VTG_ROOT/include/ZEBU_VTG.hh*), so user can create or observe Network traffic with third- party traffic generators and analyzers such as Keysight IxVerify or Spirent TestCenter Virtual.

It is common to compile the ZeBu/transactor-aware software into one of the 2 formats: namely, as a standalone binary executable, or as a shared object that can be loaded to zRci at runtime.

Note: *zRci* is a TCL based command line interface (CLI) that allows user to interact with ZeBu emulator over a tclsh-like user interface.

Standalone Binary Executable

To generate a standalone binary executable for the MII or PCS model, perform the following steps:

For MII:

```
$ cd example/ex1-zebu; make IF=mii
```

For PCS:

```
$ cd example/ex1-zebu; make IF=pcs
```

The resulting executable, named testbench, is generated at the end of the make process. Ensure that you are using GNU make 3.82 or above, and GNU GCC 6.2.0 or higher.

The source code for the executable can be found at example/ex1-zebu/testbench.cc .

Feedback

Integration Key Points

There are a few more key points to be considered during the integration:

The entry point to Virtual Ethernet is *SNPS::VSA::VTG::init( )*, which requires a Virtual Ethernet YAML configuration.

For example:

```
// pRpcPort is optional, it returns GRPC port to connect from client
bool SNPS::VSA::VTG::init(const char *yaml_file, unsigned int *const
 pRpcPort=NULL);

if (!SNPS::VSA::VTG::init("example.yml", &rpcPort))
    exit(-1);
```

To use Virtual Ethernet, you need to create a ZEMI3 manager object. You can retrieve it using the static function *ZEMI3Manager::open()*. Ensure you initialize the ZEMI3 manager first before calling *SNPS::VSA::VTG::init()*, as shown below:

```
ZEMI3Manager *zemi3_mgr = ZEMI3Manager::open("zebu.work");
  zemi3_mgr->buildXtorList();
  zemi3_mgr->init();
```

Also before calling *SNPS::VSA::VTG::init()*, create and initialize a *svt_c_runtime_cfg* object. The default *svt_c_runtime_cfg* is referenced by Virtual Ethernet / *xtor_enet_svs* later. For example:

```
ZEBU::Board *zebu_board = ZEBU::Board::getBoard();

  svt_c_runtime_cfg *m_runtime = new svt_c_runtime_cfg();
  m_runtime->set_threading_api(new svt_pthread_threading());
  m_runtime->set_platform(new svt_zebu_platform(zebu_board,1));
  svt_c_runtime_cfg::set_default(m_runtime);
```

You may start executing with the ZeBu model (that is, starting all clocks in the ZeBu emulator) after *SNPS::VSA::VTG:init()*:

```
zemi3_mgr->start();
```

At the end of the ZeBu session, terminates the process with:

```
SNPS::VSA::VTG::cleanup();
zemi3_mgr->close();
```

Dynamic Shared Object for zRci

If the user prefers to use zRci, they can generate a dynamic shared library for the MII or PCS model by following these instructions:

For MII:

```
$ cd example/ex2-zrci; make IF=mii
```

Feedback

For PCS:

```
$ cd example/ex2-zrci; make IF=pcs
```

This creates a shared object called testbench.so from the source code located at example/ex2-zrci/testbench.cc. The shared object can then be loaded into zRci at runtime with the following command:

```
zRci > testbench -load ./testbench.so
```

Integration Key Points

There are a few more key points to be considered during the integration:

- As compared to standalone binary executable, it is advised to integrate Virtual Ethernet into zRci by extending the Tcl commands of the zRci Tcl interpreter by calling `SNPS::VSA::VTG::tclInit()`.

  ```
  void *zRci_post_board_init(const TBOPTS &opts) {
      ...
      SNPS::VSA::VTG::tclInit(opts.interp);
  ```

  - The setup is completed by loading the Virtual Ethernet Tcl packages in zRci:

    ```
    zRci > lappend auto_path "$env(ZEBU_VTG_ROOT)/api/tcl/vtg1.0"
    zRci > package require ::vtg
    zRci > package require ::zebu
    ```

  Access the Virtual Ethernet Tcl Commands, under the namespace of `::vtg` and `::zebu`, from zRci hereafter.

- Choose to specify the YAML configuration with the C++ API `SNPS::VSA::VTG::init()`, or with the Virtual Ethernet Tcl command `vtg::init`. In the setup provided in $ZEBU_VTG_ROOT/example/ex2-zrci, the following Tcl command `vtg::init` is used:

  ```
  zRci > vtg::init example.yaml
  ```

- Before initializing Virtual Ethernet with a user-provided YAML configuration, either through the C++ API `SNPS::VSA::VTG::init()` or the Tcl command `vtg::init`, ensure that the default `svt_c_runtime_cfg` is setup. The default `svt_c_runtime_cfg` is referenced by the `xtor_enet_svs` transactor during initialization. It is recommended to do this in `zRci_post_board_init()`.

  ```
  void *zRci_post_board_init(const TBOPTS &opts) {
  ...
  svt_c_runtime_cfg *runtime = new svt_c_runtime_cfg();
  runtime->set_threading_api(new svt_pthread_threading());
  runtime->set_platform(new svt_zebu_platform(opts.board,1));
  svt_c_runtime_cfg::set_default(runtime);
  ```

- Include `SNPS::VSA::VTG::cleanup()` in `zRci_cleanup()`:

```
void *zRci_cleanup(const TBOPTS &opts) {
   SNPS::VSA::VTG::cleanup();
   return (NULL);
 }
```

bypass_crc_strip API

This API is used to bypass crc stripping when the data is sent to the third party traffic generator.

```
@param int xtor_id
@param bool enable/disable
```

Below is an example to bypass crc stripping for xtor 1:

```
SNPS::VSA::VTG::bypass_crc_strip(1,1)
```

A Note on CXXABI

When compiling the software testbench, set the flag `-D_GLIBCXX_USE_CXX11_ABI` to specify if the code should be compiled with the C++11 Application Binary Interface (ABI) or with an older ABI.

The choice is ZeBu version dependent because ZeBu versions earlier than S-2021.09 use CXXABI=0, whereas, ZeBu S-2021.09 and later versions use CXXABI=1. The current version of Virtual Ethernet supports only ZeBu S-2021.09 or later versions.

To use `xtor_enet_svs` transactor, compile your testbench code with `CXXABI=1` as follows:

```
$ g++ -D_GLIBCXX_USE_CXX11_ABI=1 \
-L$(ZEBU_VTG_ROOT)/libABI1 -lvtg \

-L$(ZEBU_IP_ROOT)/xtor_enet_svs/libABI1 -lxtor_enet_svs \
...
```

See the file $ZEBU_VTG_ROOT/example/ex*/Makefilefor details as other switches are required.

Feedback

Starting the ZeBu Model

As discussed in the previous section, Virtual Ethernet expects a YAML configuration that describes the user preferences at runtime. Two sample YAML configurations are available for reference:

- For Keysight IxVerify:

  `$ZEBU_VTG_ROOT/example/model.mii/common/keysight/example.yml`

- For Spirent TestCenter Virtual:

  `$ZEBU_VTG_ROOT/example/model.mii/common/stcv/example.yml`

The make process, running from the directory example/ex1-zebu and `example/ex2-zrci`, copies the YAML file for the corresponding Virtual Ethernet package to the current directory.

The make process also creates symbolic a link to `../model.<IF>/zebu/ zcui.work/ zebu.work`, where `IF` can be `mii` or `pcs`, assuming that this is the model user is going to run. If that is not the case, update the `Makefile` accordingly.

To start the ZeBu Model with the standalone binary executable, do:

```
$ cd example/ex1-zebu; ./testbench
```

To start the ZeBu Model over `zRci`, do:

```
$ cd example/ex2-zebu; zRci run.tcl
```

Users should observe the following:

- The ZeBu model is downloaded to ZeBu server.
- The YAML configuration is read by Virtual Ethernet, and the `xtor_enet_svs` transactors included in the YAML file are initialized as per the `hwconfig` setting specified. At the end of this phase, the following information is observed on the ZeBu terminal:

```
INFO: hw_top.xtor_enet_cluster[5].is400G.inst [xtor_enet_400G_svs]
 Reset deasserted  ...
INFO: hw_top.xtor_enet_cluster[0].is800G.inst [xtor_enet_800G_svs]
 Reset deasserted  ...
INFO: hw_top.xtor_enet_cluster[4].is400G.inst [xtor_enet_400G_svs]
 Reset deasserted  ...
INFO: hw_top.xtor_enet_cluster[3].is400G.inst [xtor_enet_400G_svs]
 Reset deasserted  ...
INFO: hw_top.xtor_enet_cluster[1].is800G.inst [xtor_enet_800G_svs]
 Reset deasserted  ...
INFO: hw_top.xtor_enet_cluster[2].is800G.inst [xtor_enet_800G_svs]
 Reset deasserted  ...
```

- The Virtual Ethernet RPC server is launched, listening to the rpcport specified in the YAML configuration and waiting for connection from clients.

```
INFO : ZEBU_VTG [init] RPC server: starts listening to port: 31517
```

- If Virtual Ethernet is integrated into the software testbench with `SNPS::VSA::VTG::tclInit()`, the Tcl prompt is presented.

  Note:
  At this point Virtual Ethernet has not started the workers yet. In Virtual Ethernet terminology, workers are CPU threads serving the `xtor_enet_svs` transactors. Without the workers, the `xtor_enet_svs` transactors are in idle state. This is useful because user might want to preserve CPU resources for other processes before exercising the Networking subsystem in the design under test.

SEM Example

Prerequisites:

You must add the following environment variables before you begin with compilation:

```
ZEBU_SYSTEM_DIR and FILE_CONF (specific to SEM as it is not same as the
 one used for ZeBu)
setenv FLOW SEMB
```

Building a SEM Model

Simulated Emulation (SEM) is a technology where the model build flow is integrated into the ZeBu zCui user interface while utilizing the VCS flow. This integration facilitates a quick turnaround for diagnosing initial RTL issues, such as connectivity, clock generation, and basic modeling problems.

Add the following command in the UTF file:

```
compile -sem true -sem_params
 {mode=behavior,build_flags="+vcs+initreg+random -y $ZEBU_IP_ROOT/vlog -y
 $ZEBU_IP_ROOT/vlog/vcs -y ../common/rtl $ZEBU_IP_ROOT/vlog/svt_dpi.sv"}
```

To build a SEM Model, use the following paths:

- **For MII**:

```
cd $ZEBU_VTG_ROOT/example/model.mii/sem
make
```

Feedback

- **For PCS**:

```
cd $ZEBU_VTG_ROOT/example/model.pcs/sem
make
```

Run SEM Example

Set the following environment variables before starting the test run:

```
#VTG
setenv SIMV_ZEBU_OPTIONS "-sv_lib $ZEBU_VTG_ROOT/libABI1/libvtg -sv_lib
 $ZEBU_VTG_ROOT/libABI1/libvtg-common
  -sv_lib $ZEBU_IP_ROOT/lib/libxtor_enet_svs -sv_lib
 $ZEBU_IP_ROOT/lib/libZebuXtor"
```

Following are the changes in testbench.cc:

```
//We need to stat our testbench for simulator platform
//and we need to use svt_systemverilog_threading instead
//of svt_pthread_threading.
void *zRci_post_board_init(const TBOPTS &opts) {
  ...
- runtime->set_threading_api(new svt_pthread_threading());
- runtime->set_platform(new svt_zebu_platform(opts.board, 1));
+ runtime->set_platform(new svt_simulator_platform());
+ runtime->set_threading_api(new svt_systemverilog_threading());
  ...
  return NULL;
}
```

- **For MII**: `cd example/ex-sem; make IF=mii`
- **For PCS**: `cd example/ex-sem; make IF=pcs`

```
zRci run.tcl
```

The SEM example is located in the Example directory. Use the following path:

```
$ZEBU_VTG_ROOT/example/ex-sem
```

To run SEM example, refer `$ZEBU_VTG_ROOT/example/README`.

Limitation (s)

Generic SEM limitation

Clock support - RTL Clocks (#delays and clockDelayPort).

Only works in zRci mode.

Virtual Ethernet

Sync and staggered start on single/multiple host is not supported.

The following APIs are not supported in client connection mode, which is used for interactive sessions. However, these APIs function seamlessly in batch mode or when invoked from C++ testbench.

- vtg::system::
- vtg::xtor::ll
- vtg::xtor

Following is the list of ZEBU APIs that are not supported:

- zebu::runmgr
- zebu::signal
- zebu::fwc
- zebu::wavefile
- zebu::tcleval
- zebu::version - ZeBu version not supported, others are supported
- zebu::sniffer

Port rate and bandwidth are not ensured in SEM mode which is a transactor limitation.

NTGB Example

The NTGB example is located inside the Example directory, similar to ex1-zebu and ex2-zrci. The following sections explain how to build the NGTB VTG library and run it using NGTB.

Feedback

Building the VTG NGTB Shared Library (NGTB Testbench)

To build the VTG NGTB Shared Library, do the following:

1. Go to the directory:

   ```
   cd ${ZEBU_VTG_ROOT}/example/ngtb-example
   ```

2. Build the library:
   - For MII, use `make IF=mii`
   - For PCS, use `make IF=pcs`

Running the NGTB Example

To run the NGTB example, use `./ngtb_run.sh` command.

Starting the Virtual Ethernet Process

When users are ready to start testing with the Networking subsystem in the DUT, start the Virtual Ethernet process with one of the 2 methods:

- From zRci
- From a remote Tcl client

From zRci

First, make sure that Virtual Ethernet Tcl packages are loaded:

```
zRci > lappend auto_path "$env(ZEBU_VTG_ROOT)/api/tcl/vtg1.0"
zRci > package require ::vtg
zRci > package require ::zebu
```

To start the Virtual Ethernet process, do:

```
zRci > vtg::system::start
```

From a Remote Tcl Client

From a new terminal, start a Tcl shell (version 8.5 or above). You will be presented with a tclsh prompt:

```
tclsh8.5
tclsh >
```

In this Tcl shell, load the Virtual Ethernet Tcl packages and make a connection to the Virtual Ethernet RPC server running from the host $ZEBU_HOST over the TCP port specified as rpcport in the YAML configuration:

```
tclsh > lappend auto_path "$env(ZEBU_VTG_ROOT)/api/tcl/vtg1.0" tclsh >
 package require ::vtg
tclsh > package require ::zebu
tclsh > vtg::connect $::env(ZEBU_HOST):$::env(RPC_PORT)
```

After the Tcl client is connected to the RPC server, the following is displayed on the ZeBu terminal:

```
INFO: ZEBU_VTG [] rpc client vgzeburt446 is now connected
```

Now send the following command to start the Virtual Ethernet process:

```
tclsh > vtg::system::start
```

It is recommended to execute Virtual Ethernet Tcl commands from a remote Tcl client for a few reasons:

- The Virtual Ethernet RPC server can handle multiple client connections simultaneously. If one client gets stuck (for instance, due to an infinite loop), you can simply terminate that client and reconnect to the RPC server using a new client.
- The zRci prompt is always available and does not get locked up by Virtual Ethernet Tcl commands. You can use the zRci prompt to troubleshoot problems that may cause the system to stop responding, such as checking if the ZeBu clocks are stopped for any reason.
- Any logging information from Tcl commands are shown on the Tcl client's screen, not on the zRci screen. This way, the zRci prompt is not overwhelmed with debugging messages and remains usable.

Breakdown of the Virtual Ethernet Process

The Tcl command vtg::system::start can actually be broken down into multiple substeps:

- For Keysight IxVerify
  - If the section ixverify in the YAML configuration is specified, Virtual Ethernet will start a vhost server.
  - If the YAML configuration also specifies virtual load modules, Virtual Ethernet will launch these virtual machines based on the details provided in the YAML file.

- Start the IxVerify sync server on the ZeBu host. This sync-server is required to enable the synchronous start feature in IxVerify.
- Start the worker manager, which manages and dispatch all the workers afterwards. Workers start checking if there are Ethernet packets to be exchanged between virtual ports of Virtual Load Modules and `xtor_enet_svs` transactors.

- For Spirent TestCenter Virtual
  - If the section `stc` in the YAML configuration is specified, Virtual Ethernet starts a vhost server.
  - If the YAML configuration also specifies virtual chassis, Virtual Ethernet launches these virtual machines based on the details provided in the YAML file.
  - Start the worker manager, which manages and dispatch all the workers afterwards. Workers start checking if there are Ethernet packets to be exchanged between virtual ports of STCv Chassis and `xtor_enet_svs` transactors.

The Virtual Ethernet process can be shut down at any time using another Tcl command:

```
tclsh > vtg::system::stop
```

The process can also be restarted again if user issues `vtg::system::start` after.

Expected Log Message from ZeBu Terminal

After `vtg::system::start` is executed, user should observe these debug messages from the ZeBu terminal:

```
INFO: ZEBU_VTG [WorkerMgr] add xtor_enet_svs#1 to worker 0
INFO: ZEBU_VTG [WorkerMgr] add xtor_enet_svs#2 to worker 0
INFO: ZEBU_VTG [WorkerMgr] add xtor_enet_svs#3 to new worker 1
INFO: ZEBU_VTG [WorkerMgr] add xtor_enet_svs#4 to worker 1
INFO: ZEBU_VTG [WorkerMgr] add xtor_enet_svs#5 to worker 1
INFO: ZEBU_VTG [WorkerMgr] Checkout vTG licenses successfully
DEBUG: hw_top.xtor_enet_cluster[0].is800G.inst [xtor_enet_800G_svs]
 XTOR_ENET_SVS 800G :: Configuring : ENET_SPEED with v DEBUG :
 hw_top.xtor_enet_cluster[0].is800G.inst [xtor_enet_800G_svs]
 XTOR_ENET_SVS 800G :: XTOR_ID == 0
DEBUG: hw_top.xtor_enet_cluster[0].is800G.inst [xtor_enet_800G_svs]
 XTOR_ENET_SVS 800G :: Configuring : ENET_DATA_CNT_EMP
...
INFO: ZEBU_VTG::worker0 [tx_serviceloop] start with 3 members(0 1 2 ),
 --no-timestamp=0
...
INFO: ZEBU_VTG::worker1 [tx_serviceloop] start with 3 members(3 4 5 ),
 --no-timestamp=0
```

Feedback

If ixverify or stcv sections are presented in the YAML configuration provided, and port mapping happens successfully:

```
INFO : ZEBU_VTG [VirtMgr] vhost socket is now opened
INFO : ZEBU_VTG [vmStart] chas1: virsh desc: virbr=virbr0,
 eth=192.168.122.128
INFO : ZEBU_VTG [vmStart] start chas1.card11 now ...
hdr version 3.6
INFO : ZEBU_VTG [vdevMap] Virtual port 0 {0xc0a87a80,11,1} is mapped to
 xtor_enet_svs#0
INFO : ZEBU_VTG [vdevMap] 1 port(s) now mapped
INFO : ZEBU_VTG::xtor_enet_svs#0 [vhostDequeue] reset dashboard
hdr version 3.6
INFO : ZEBU_VTG [vdevMap] Virtual port 1 {0xc0a87a80,11,2} is mapped to
 xtor_enet_svs#1
INFO : ZEBU_VTG [vdevMap] 2 port(s) now mapped
hdr version 3.6
INFO : ZEBU_VTG [vdevMap] Virtual port 2 {0xc0a87a80,11,3} is mapped to
 xtor_enet_svs#2
INFO : ZEBU_VTG [vdevMap] 3 port(s) now mapped
INFO : ZEBU_VTG::xtor_enet_svs#1 [vhostDequeue] reset dashboard
INFO : ZEBU_VTG::xtor_enet_svs#2 [vhostDequeue] reset dashboard
...
```

8

Third Party Applications and Libraries

You are recommended to install third party applications and libraries to make testing with Virtual Ethernet Tcl commands easier.

For more information, see the following topics.

sshpass

SSHpass is a tool that enables non-interactive SSH login to remote servers. It is used to automate SSH connections by bypassing the need to enter a password each time. This tool is used to automate the setup and launching of IxVerify Virtual Chassis.

If SSHpass is not available in your system, follow these steps to download and install the sshpass RPM package without root permission:

- Find the sshpass RPM package for your Linux distribution and architecture. You can search for it on the Internet or on the official package repository for your distribution.
- Download the RPM package to your local system.
- Install the RPM package using the `rpm2cpio` command. For example, to install the package in ~/local, run:

```
$ mkdir ~/local
$ rpm2cpio sshpass-<version>.rpm | cpio -idmv -D ~/local
```

- Finally, add the directory ~/local/bin to PATH environment variable.

```
# !/bin/bash
  $ export PATH=~/local/bin:${PATH}
```

Tclreadline Library

It is recommended that you install the Tclreadline library as it provides command-line history and tab completion capabilities to the Tcl interpreter, which can make working with the command line interface more efficient and convenient.

Feedback

Follow these steps to download and install the Tclreadline RPM package:

1. Find the Tclreadline RPM package for your Linux distribution and architecture. You can search for it on the Internet or on the official package repository for your distribution.
2. Download the RPM package to your local system.
3. Install the RPM package using the `rpm2cpio` and `rpm` command. If you do not have root access, use the `--prefix` option to install the package to directory where you have write permissions. For example, to install the package in /home/vtg/tclreadline, run:

```
$ rpm2cpio tclreadline.rpm | cpio -idmv
$ rpm -ivh --prefix=/home/vtg/tclreadline
 tclreadline-2.3.8-1.x86_64.rpm
```

4. Finally, add the Tclreadline binaries to `TCLLIBPATH` environment variable.

```
# !/bin/bash
$ export TCLLIBPATH=/home/vtg/tclreadline/lib64/tcl8.5
```

9

Using Python for Virtual Ethernet

Virtual Ethernet supports Python clients. However, Python client offers a limited set of APIs. The following topics describe the steps to setup, install, and run Virtual Ethernet on a Python client:

- Setup and Install
- Python APIs

Setup and Install

Prerequisites

- Ensure `python3.7 or above` is installed.
- Ensure python calls `python3`.

The steps to setup, install, and use a Python virtual environment are as follows:

1. Run the `setup.csh` script:

   ```
   $ cd vtgPkg/example/tests/python
   $ source ./setup.csh
   ```

   This script helps:

   - create a virtual environment called `vtgenv` under the current directory.
   - activate the virtual environment.
   - install all required python packages in this virtual environment.
   - generate intermediate files and setup environment variables required by Virtual Ethernet.

2. Activate the virtual environment `vtgenv` created in step 1. To do so, run the following command:

   ```
   $ source vtgenv/bin/activate.csh
   (vtgenv)
   ```

   The prompt (`vtgenv`) indicates the Python virtual environment is currently active.

Feedback

To exit the virtual environment (`vtgenv`), type `deactivate`.

3. Start Python by typing `python` at the (`vtgenv`) prompt as follows:

```
(vtgenv) python
```

The following message is displayed indicating that the Python APIs can be used:

```
Python 3.11.0 (main, Nov 10 2022, 08:24:18) [GCC 8.2.0] on linux
Type "help", "copyright", "credits" or "license" for more
information.
>>>
```

4. To use the Python Virtual Ethernet APIs, you need to first import them. To do so, run the following command:

```
>>> from Vtg.api import *
```

You can run the following examples at the Python prompt `>>>` to ensure that Python virtual environment is all set to run the VTG Python APIs.

```
>>> vtg.test.log ('INFO', 'Helloworld!');
>>> exec(open("./speedtest.py").read())
```

To explore all namespaces and functions defined in Virtual Ethernet, type `>>> dir()`.

To connect a non-default grpc port, type the following commands:

```
#tcp_port = 35611
>>> vtg.connect("localhost",tcp_port)
```

To start vtg, type `>>> vtg.system.start()`.

Python APIs

The following sections lists the Python APIs that are currently supported:

- Transactor Specific High-Level Commands (implemented by Virtual Ethernet)
- Transactor Specific Low-Level Commands (implemented by xtor_enet_svs)
- Logging-Related Commands
- System Initialization and Client Connection Commands

Feedback

Transactor Specific High-Level Commands (implemented by Virtual Ethernet)

This section describes the following commands:

- vtg.xtor.get_pcsstatus()

vtg.xtor.get_pcsstatus()

Description

Gets PCS status from the Ethernet transactor hardware

Syntax

```
vtg.xtor.get_pcsstatus(cls, iXid:int , strCmd:str = "")
```

Arguments

```
vtg.xtor.get_pcsstatus FIELD_LIST [OPTION] XID
```

`FIELD_LIST`: See `XTOR_ENET_SVS.pcsStatusDbStruct` in `$ZEBU_IP_ROOT/include/xtor_enet_svs_defines.hh` for supported status.

Note:

Most fields defined in `pcsStatusDbStruct` has a valid bit. If that field is invalid, this procedure returns -1.

The fields '`first_am_detect`' and '`second_am_detect`' are exceptional, as they have a bitmask instead of a valid bit. You might retrieve its bitmask by specifying a string `<field>.bitmask`. See the example for additional details.

`[OPTION]`: Is specified as `--as-array`. If specified, an associate array is returned. Otherwise, a value list is returned.

`XID`: Specifies the transactor ID.

Examples

Read PCS lock status and `blkCntLn0` of transactor 0

```
> vtg.xtor.get_pcsstatus(0,"pcsLockStatus blkCntLn0")
1 1000
```

Read PCS lock status from transactor 0's hardware before reporting

```
> vtg.xtor.get_pcsstatus(0,"pcsLockStatus")
1
```

Feedback

Read RS-Decorder symbol error count from xtor 2, which find this field invalid

```
 > vtg.xtor.get_pcsstatus(2,"rsdecSymerrCount")
-1
```

Read blkCntLnx from xtor 2 as an array

```
> vtg.xtor.get_pcsstatus(2,"blkCntLn0 blkCntLn1 --as-array")
1004
```

Read first AM detect value and its bitmask from transactor 3

```
> vtg.xtor.get_pcsstatus(3,"first_am_detect first_am_detect.bitmask")
254 65535
```

Transactor Specific Low-Level Commands (implemented by xtor_enet_svs)

This section describes the following commands:

- vtg.xtor.ll.print_dashboard()
- vtg.xtor.ll.set_config()
- vtg.xtor.ll.get_config()

vtg.xtor.ll.print_dashboard()

Description

Calls the `printDashboard()` API of transactor.

Syntax

```
vtg.xtor.ll.print_dashboard(cls, iXid:int , strCmd:str = "")
```

Arguments

```
XID [OUTPUT]
```

`[OUTPUT]`: Instructs transactor where to display its dashboard.

`XID`: Specifies the transactor ID.

Examples

Instructs xtor1 to display its dashboard to stdout:

```
> vtg.xtor.ll.print_dashboard(1)
```

Feedback

Instructs xtor0 to display the dashboard to the `x0_db.txt` file:

```
> vtg.xtor.ll.print_dashboard(0,"x0_db.txt")
```

vtg.xtor.ll.set_config()

Description

Calls the `setConfig()` API of transactor with the given CFG and VALUE. Returns 1 if succeeds; 0 otherwise.

Syntax

```
vtg.xtor.set_config(cls, iXid:int , strCmd:str = ""):
```

Arguments

```
XID,"CFG=VALUE"
```

`CFG=VALUE`: Instructs transactor where to display its dashboard.

`XID`: Specifies the transactor ID.

Examples

Instruct transactor with `xid=3` to set `ENET_LPBK_EN` to 0

```
> vtg.xtor.ll.set_config(3,"ENET_LPBK_EN=0")
```

```
return sv.zebu_cmd(ZEBU_VTG.OPC_XTOR_SET_CONFIG, str(iXid) +' '+ strCmd)
```

vtg.xtor.ll.get_config()

Description

Calls the `getConfig()` API of transactor with the given CFG. If successful, prints the CFG value to the stdout.

Syntax

```
vtg.xtor.get_config(cls, iXid:int , strCmd:str = "")
```

Arguments

```
XID,"CFG"
```

`CFG`: Instructs transactor to display CFG to the stdout.

`XID`: Specifies the transactor ID.

Examples

Instruct transactor with `xid=3` to get `ENET_LPBK_EN`

Feedback

```
> vtg.xtor.ll.get_config(3,"ENET_LPBK_EN")
```

Logging-Related Commands

This section describes the following commands:

- vtg.logger.new()
- vtg.logger.list()
- vtg.logger.msg()

vtg.logger.new()

Description

Creates a new logger.

Syntax

```
vtg.logger.new(cls, strCmd:str = "") -> (bool,str)
```

Arguments

"NEW_LOGGER"

NEW_LOGGER: Creates a new logger with the specified name.

Examples

Create a new logger with the name, NEW_LOGGER

```
vtg.logger.new("NEW_LOGGER")
```

vtg.logger.list()

Description

Returns the list of loggers.

Syntax

```
vtg.logger.list(cls, strCmd:str = "") -> (bool,str)
```

Examples

List existing loggers.

```
vtg.logger.list()
```

Feedback

vtg.logger.msg()

Description

Logs debug messages.

Syntax

```
vtg.logger.msg(cls, strLogger:str = "", strLog:str = "") -> (bool,str)
```

Arguments

```
NAME msg
```

`NAME`: Specifies the name of the logger.

`msg`: Specifies the messages to be written

Examples

```
vtg.logger.msg("logger_name", "message")

return sv.zebu_cmd(ZEBU_VTG.OPC_LOGGER_MSG,toStr(strLogger,strLog))
```

System Initialization and Client Connection Commands

This section describes the following commands:

- vtg.connect()
- vtg.vtgHelp()
- vtg.system.start()
- vtg.system.stop()
- vtg.system.workermgr()

vtg.connect()

Description

Presets host name and port number of the vTG/rpc server.

Virtual Ethernet has a built-in server that client applications, either running locally or remotely, can connect to. This command specifies the host name and TCP port number of such server.

TCP port number of the server is specified as 'rpcport' in the user provided YAML configuration. If 'rpcport' is not specified, Virtual Ethernet finds a free TCP port, displays the value to the stdout, and starts the server with that TCP port.

Feedback

Syntax

```
vtg.connect(cls,strHost:str = "localhost",iPort:int = 31517)->
 (bool,str):
```

vtg.vtgHelp()

Description

Presets host name and port number of the vTG/rpc server.

Virtual Ethernet has a built-in server that client applications, either running locally or remotely, can connect to. This command specifies the host name and TCP port number of such server.

TCP port number of the server is specified as 'rpcport' in the user provided YAML configuration. If 'rpcport' is not specified, Virtual Ethernet finds a free TCP port, displays the value to the stdout, and starts the server with that TCP port.

Syntax

```
vtg.vtgHelp(listapi:str = None, helpage: bool = False) :
```

Examples

Print all APIs

```
vtgHelp('')
```

Print all APIs with help page

```
vtgHelp('',1)
```

Print APIs under vtg.xtor and recursively

```
vtgHelp('vtg.xtor')
```

Print APIs under vtg.xtor and recursively with help page

```
vtgHelp('vtg.xtor',1)
```

Print for full documentation

```
help('api_name')
help(vtg.xtor.ll.get_config)
```

vtg.system.start()

Description

Starts worker threads and virtual machines. Use this command to tell Virtual Ethernet running in the remote ZeBu runtime host to start virtual machines, worker manager

Feedback

and workers, and any additional processes as specified in the YAML file provided to `vtg::init or SNPS::VSA::VTG::init` API.

This command starts the following processes:

- Virtual manager and virtual machines, which are responsible for generating and receiving Ethernet traffic.
- Worker manager and workers, which are responsible for exchanging Ethernet frames between virtual machines and transactors.
- IxVerify sync server if the traffic generator used is IxVerify from Keysight

Syntax

```
vtg.system.start()
```

vtg.system.stop()

Description

Stops worker threads and virtual machines. Use this command to tell Virtual Ethernet to stop virtual machines, worker manager and workers, and any processes that has been started previously.

Syntax

```
vtg.system.stop(cls, vendor:str = "") :
```

vtg.system.workermgr()

Description

Stops worker threads and virtual machines. Use this command to tell Virtual Ethernet to stop virtual machines, worker manager and workers, and any processes that has been started previously.

Syntax

```
vtg.system.workermgr(cls, strLog:str = "") :
```

Arguments

`[OPTIONS]`

`Init`: Just initialize worker manager and all xtors. Do not start workers.

`start [--no-timestamp]`: Start worker manager, which initializes all worker threads and dispatches them to their respective tasks. If "--no-timestamp" is specified, skip calling `getCycleCount()` and `getTxTimestamp()` periodically. Internal use only.

`stop`: Stop all workers first, then worker manager.

`is_running`: Return 1 if worker manager is running; 0 otherwise.

`get_wid`: Return a list of worker IDs.

Returns

For the command `is_running`, return 1 if the worker manager is running, 0 otherwise. For start and stop command, no return value is provided.

Examples

```
vtg.system.workermgr("start --timestamp")
vtg.system.workermgr("stop")
vtg.system.workermgr("init")
vtg.system.workermgr("is_running")
vtg.system.workermgr("get_wid")
vtg.system.workermgr("start")
```